Structure in the Recurrence

Conditioning Genomic Foundation Model Pretraining on 3D Chromatin Contacts

Project technical record

Compiled August 31, 2026

No experimental results appear in this document. No model has been trained. The abstract is a proposal abstract. Every number reported comes from a data-processing, parameter-accounting or unit-test script in the repository, never from a training run. The working rule for this project is that no figure reaches a draft unless it exists first in a logged output file.

Abstract

Genomic foundation models are pretrained on DNA sequence alone, yet much of the regulatory logic they are asked to predict is carried by the genome's three-dimensional folding, which determines which enhancers can reach which promoters. Existing work combining sequence with chromatin contact data does so after the sequence representation is already fixed: CHROME trains an encoder on individual loci and only then adds a graph-attention layer over Hi-C contacts, and Evo2HiC distils a frozen Evo 2 into a compact encoder using Hi-C as a contrastive alignment target. In both, contact data shapes which learned features are combined, never which features are learned. We propose instead to place chromatin structure inside the pretraining computation. Contact-derived features modulate the state-transition dynamics of a bidirectional Mamba backbone, so Hi-C signal changes how far information propagates along the sequence rather than only what each position encodes. The mechanism adds 1.70% to parameter count over a matched sequence-only baseline. We evaluate whether a structure-shaped representation transfers—a question no sequence-to-structure model has been asked, as a forward-citation sweep over 318 papers citing Akita and Orca confirms—using long-range regulatory and variant-effect tasks together with a perturbation-sensitivity protocol on which Evo 2 has been reported to fail. Controls include structure shuffled at pretraining time and a distance-matched rewiring that tests whether any apparent structural signal is genomic distance in disguise.

Keywords: genomic foundation models; 3D genome organisation; Hi-C; topologically associating domains; state-space models; Mamba; self-supervised pretraining; CTCF; variant effect prediction; representation transfer.

Contents

1 The problem

An enhancer can sit 10^5 to 10^6 base pairs from the promoter it regulates. Nothing in the linear arrangement of the sequence explains how one reaches the other. The genome resolves this by folding: a ring-shaped cohesin complex extrudes a loop of DNA until it stalls at a pair of CTCF binding sites, and the resulting loop places the enhancer physically against the promoter.

The stalling condition matters for what follows. Cohesin halts only when the two CTCF motifs are *convergent*—pointing toward each other. Motif presence is insufficient; orientation decides. A sequence-only model sees the motif but receives no training signal that makes orientation pairing across a 500 kb gap consequential.

Three structures recur in the analysis. A *topologically associating domain* (TAD) is a region that contacts itself far more than its surroundings; enhancers act almost exclusively within their own TAD, so boundaries determine reachability. A *loop* is a point-to-point contact between two anchors. An *A/B compartment* is a coarse partition into open, active and closed, inactive regions.

2 Related work and the gap

Work	Seq.	Struct.	Structure enters at	Needed at inference
HiCFoundation (2026)	no	yes	pretraining	yes
Hi-Cformer (2025)	no	yes	pretraining	yes
Evo 2 (2026)	yes	**no**	—	no
Caduceus (2024)	yes	**no**	—	no
GraphReg (2022)	feat.	yes	supervised, downstream	yes
CHROME (2026)	yes	yes	**stage 2, post-hoc**	**yes**
Akita / Orca	yes	target	supervised target	no
Evo2HiC (2025)	yes	yes	contrastive distillation	no
This project	**yes**	**yes**	**self-supervised pretraining**	**target: no**

Table 1: Where 3D structure first influences the model. The bottom row is unoccupied in the current literature.

2.1 The two nearest competitors

CHROME (Ye et al., *Briefings in Bioinformatics* 27(4):bbag360, 2026) filters Hi-C against a self-avoiding polymer null model simulated over 5×10^5 chains, retaining 1.8–6% of contacts, then runs two graph-attention layers over signal-centred subgraphs with a 4 Mb receptive field at 5 kb bins. Training has two stages: the encoder is trained on centre nodes alone until validation saturates, and only afterwards are early layers frozen and the graph module attached. Supervision is 751 ENCODE assays. Structure never influences what the sequence encoder learns, and the contact graph is required at inference.

Evo2HiC (Fang et al., bioRxiv 10.1101/2025.11.18.689171) distils a frozen Evo 2 (7B) into a 3.6M-parameter CNN using two SigLIP contrastive objectives, the second aligning the student to Hi-C patch embeddings. Structure genuinely shapes a sequence encoder, but by aligning output embeddings, with no objective over nucleotides. A keyword count over the full preprint returns zero occurrences of *ClinVar*, *pathogenic*, *eQTL*, *variant*, *masked*, *MLM*, *Mamba* and *state space*.

Their third stated limitation is that they froze Evo 2 because fine-tuning it is computationally hard, and that they intend to develop strategies for adapting it with Hi-C data—this project's question, named as work they have not done.

2.2 Two external findings that shaped the plan

Lee (arXiv:2604.07196, April 2026) probed Evo 2 (7B) on 231 TAD boundary regions and 120 convergent CTCF loops. Boundary deletions were penalised *less* than GC- and size-matched random controls (paired Wilcoxon $p = 0.405$); CTCF inversions and deletions likewise ($p = 0.021$, $p = 0.006$, in the wrong direction). Generated loops reached median enrichment 0.054 against a reference of 0.388. The paper concludes Evo 2 "has learned local CTCF grammar but misses higher-order 3D organization" and prescribes bidirectional architectures with explicit 3D contact inputs, citing Caduceus. This substantially defuses failure mode F3 (Section 6): if a 7B-parameter model with a 1M-token context cannot infer TAD-scale structure from sequence, a 7.7M-parameter model will not either.

DNALongBench (*Nature Communications* 16(1):10108, 2025) provides five long-range tasks including enhancer–target interaction, eQTL and contact-map prediction, already benchmarking Caduceus-Ph and Akita. Adopting it for evaluation removes the objection that tasks were chosen to flatter. Calibration: contact-map prediction is the hardest task in the suite, best reported correlation 0.233, by Akita.

2.3 The negative result the contribution rests on

A forward-citation sweep over 318 papers—118 citing Orca, 200 citing Akita—found no work that takes either model's learned representations and evaluates them on a task other than predicting folding. Their variant applications all run the model twice, on reference and mutant sequence, and compare predicted contact maps. Internal embeddings are never extracted, frozen and probed, or fine-tuned into another head.

3 Method

3.1 Baseline recurrence

Per layer and direction, with channel index i and state index n:

$$(z, v) = \mathrm{split}(W_{\mathrm{in}} u_t) \tag{1}$$
$$x_t = \mathrm{SiLU}(\mathrm{Conv1d}(v)_t) \tag{2}$$
$$(\delta'_t, B_t, C_t) = \mathrm{split}(W_x x_t) \tag{3}$$
$$\Delta_t = \mathrm{softplus}(W_\delta \delta'_t + b_\delta) > 0 \tag{4}$$
$$A_{i,n} = -\exp(A^{\log}_{i,n}) < 0 \tag{5}$$
$$\bar{A}_{t,i,n} = \exp(\Delta_{t,i} A_{i,n}) \in (0, 1) \tag{6}$$
$$h_{t,i,n} = \bar{A}_{t,i,n}\, h_{t-1,i,n} + \Delta_{t,i} B_{t,n} x_{t,i} \tag{7}$$
$$y_{t,i} = \textstyle\sum_n C_{t,n} h_{t,i,n} + D_i x_{t,i} \tag{8}$$

3.2 The structural modification

Let $s_t \in \mathbb{R}^{d_s}$ be the structural context at position t, derived from Hi-C. Two terms are added:

$$\Delta_t = \mathrm{softplus}\big(W_\delta \delta'_t + b_\delta + W_{\Delta s} s_t\big) \tag{9}$$
$$p_t = \mathrm{softplus}\big(w_g^\top s_t + b_g\big) \;\geq 0 \tag{10}$$
$$\bar{A}_{t,i,n} = \exp\left(\Delta_{t,i} A_{i,n} - p_t\right) \tag{11}$$

The first retunes the memory length of each channel. The second is a non-negative damping that only shortens memory: at a TAD boundary the model can raise it and cut the state.

Why this is not an input feature. Appending s_t to the input would reach only $\bar{B}_t x_t$, the term governing how much of the current token enters. It cannot alter $\bar{A}_t$, which governs what survives. Because Δ enters $\bar{A}$ inside an exponential, structure here sets the effective memory horizon

$$\tau_{t,i} = \frac{-1}{\Delta_{t,i} A_{i,n} - p_t} \quad \text{(tokens)}, \tag{12}$$

that is, how *far* information travels rather than what is encoded. This is the class of constraint that contrastive alignment on output embeddings cannot express, and it is the entire novelty claim.

Relationship to hard state resets. As $p_t \to \infty$, $\bar{A}_t \to 0$ and $h_t = \bar{B}_t x_t$: a hard reset. The mechanism is the soft, learned relaxation of TAD-conditioned state resetting, which is recorded as future work.

Initialisation. $W_{\Delta s} = 0$, $w_g = 0$, $b_g = -4$. The model is then numerically indistinguishable from the baseline at step zero, so any divergence is attributable to learned structural signal. Zero initialisation of $W_{\Delta s}$ does not block learning, since $\partial L / \partial W_{\Delta s} = (\partial L / \partial \Delta)\, s_t^\top$.

3.3 Parameter accounting

Computed by instantiating both models and summing parameter tensors (`scripts/param_accounting.py`, `scripts/test_model.py`).

Configuration	Total	Added	Δ
Sequence-only baseline	7,725,312	—	—
$d_s = 2$, with encoder	7,758,354	33,042	+0.428%
$d_s = 8$, with encoder	7,856,952	131,640	+1.704%
$d_s = 8$, **no encoder** (recommended)	7,856,672	131,360	+1.700%

Table 2: All configurations sit inside the 5% matched-parameter budget.

4 Implementation

4.1 Scan backends

The selective scan has two implementations, selected by device.

The reference implementation is a sequential loop over positions: correct, differentiable, and what the CPU unit tests run at small L. It is not viable at $L = 32{,}768$, which would require $32{,}768 \times 16 \times 2$ Python iterations per forward pass.

Production requires a fused kernel, and the two structural terms differ sharply in how they cooperate with the stock one. **The Δ bias is free**: `mamba_ssm`'s `selective_scan_fn` takes `delta` as an argument, so the bias is added before the call. **The permeability term p_t is not expressible** through that interface: it needs $\bar{A} = \exp(\Delta A - p)$, and folding p into Δ would require $\Delta' = \Delta + \log(g)/A_n$, which depends on the state index n. A single per-channel Δ cannot produce a decay uniform across n.

A custom Triton kernel was therefore written (`src/chromfm/scan_triton.py`), carrying p through both forward and backward. It uses one program per (batch, channel) pair, checkpoints

the state at chunk boundaries, and in the backward replays each chunk from its checkpoint before running the adjoint recurrence, keeping memory at $O(BD\,\frac{L}{\text{chunk}}N)$ rather than $O(BDLN)$.

The Triton kernel has never been executed. It was written on a CPU-only machine with no CUDA device and no Triton install. The recurrence and its adjoint were derived by hand and the code is syntactically complete, but nothing is numerically confirmed. `scripts/validate_kernel.py` must be run on the GPU box before any training. A scan with a subtly wrong gradient produces a smooth, plausible loss curve while training the wrong model.

Both arms must use the same backend, or the matched-compute comparison is confounded. The baseline therefore runs on this scan too, even though it never supplies p.

4.2 Dataset layer

Window		32,768 bp
Train stride		16,384 (50% overlap)
Evaluation stride		32,768 (no overlap)
Train windows		5,422
Validation windows	273	(chr9:120,029,184–128,974,848)
Test windows	242	(chr9:130,023,424–137,953,280)
Dropped	1,054 on N fraction, 799 on structural coverage, 126 buffer	

Table 3: Window index. Verified: zero training windows touch either held-out region; both evaluation splits are fully contained with no internal overlap.

Window length follows from the memory-horizon measurement rather than convention: the 16-layer relayed horizon is roughly 16,000 tokens, so 32,768 already exceeds what the stack can span.

Three policies are implemented in the dataset builder rather than in either training script, so the two arms cannot diverge on them:

(i) ϕ **is interpolated, not stepped.** Linear interpolation between bin centres. Measured inside one window, a step function yields 7 distinct values per feature; interpolation yields 32,757. Seven values across a window is the precondition for failure mode F4.

(ii) **N has its own token and a budget.** 12.00% of chr9 is unresolved; windows above 10% N are dropped.

(iii) **Invalid bins are masked, never NaN.** A position is structurally valid only when both flanking bins are usable. Invalid positions receive $\phi = 0$, the standardised mean, plus a validity flag. A NaN reaching $W_{\Delta s}s_t$ would poison the entire recurrence.

Reverse-complement augmentation reverses and complements the tokens, then reverses ϕ and negates its two antisymmetric coordinates (`directionality_2Mb`, `upstream_mass_frac`).

5 Pilot data

5.1 Provenance

Source is 4D Nucleome experiment set **4DNES3JX38V5**: in situ Hi-C on GM12878, MboI with bio-dATP, Lieberman Aiden lab, published as Rao et al. 2014 (PMID 25497547), aligned to GRCh38. The multi-resolution contact matrix (4DNFIXP4QG5B) is 27.4 GB and is never downloaded; the pipeline reads a near-diagonal band directly over HTTP range requests in six and a half minutes. Boundary calls, insulation and compartment tracks are mirrored as independent validation targets, not as model inputs. Sequence is Ensembl release 113; annotation is GENCODE release 47.

5.2 Measured properties

chr9 length, from the contact matrix	138,394,717 bp
chr9 length, from the Ensembl FASTA	138,394,717 bp
Bins at 5 kb	27,679
Band entries retained	7,108,483
Band occupancy	0.6451
Bins with no balancing weight	19.31%
Unresolved sequence (N)	12.00%
Bins with a complete feature vector	21,519 (77.74%)

The two identical lengths are the first coordinate-alignment check. A second, stronger one emerged unplanned: the pipeline returned nothing for chr9 between 48 and 58 Mb. That is the centromere and the 9q12 heterochromatic block, which GRCh38 represents as a gap. The sequence file is 100% unresolved from approximately 46 to 60 Mb, and the contact matrix has no usable bins from 43.27 to 60.52 Mb. Two files from different institutions placing the same feature at the same coordinates is stronger evidence than a matching chromosome length.

5.3 Structural features

Index	Feature	Window	Under reverse complement
0–2	insulation score	100, 250, 500 kb	symmetric
3	directionality index	2 Mb	**antisymmetric**
4	log contact density	2 Mb	symmetric
5	upstream mass fraction	2 Mb	**antisymmetric**
6	short-to-long range ratio	100 kb / 2 Mb	symmetric
7	A/B compartment eigenvector	250 kb, GC-oriented	symmetric

Table 4: The symmetry column is load-bearing: failure mode F7 is the case where features 3 and 5 cancel between the forward and reverse passes.

5.4 Validation against independent tracks

Our feature	Against 4DN's own track	Pearson r
Insulation, 100 kb window	4DNFIBMOGOZC	+0.9969
Insulation, 250 kb window	4DNFIBMOGOZC	+0.7802
Insulation, 500 kb window	4DNFIBMOGOZC	+0.4895
A/B compartment eigenvector	4DNFILYQ1PAY	+0.9759

Table 5: The 100 kb window reproduces 4DN's track almost exactly, identifying their diamond window and confirming the implementation. Declining correlation at larger windows is the expected consequence of comparing different window sizes against a fixed reference.

A TAD and a loop are both visible in the processed data. The loop lies at chr9:136,485,000 paired with 136,625,000, 140 kb apart, at 7.5 times its distance-matched expectation, with six CTCF ChIA-PET read pairs from an independent assay at the same anchors. Its upstream anchor falls on *NOTCH1* (chr9:136,494,433–136,546,048).

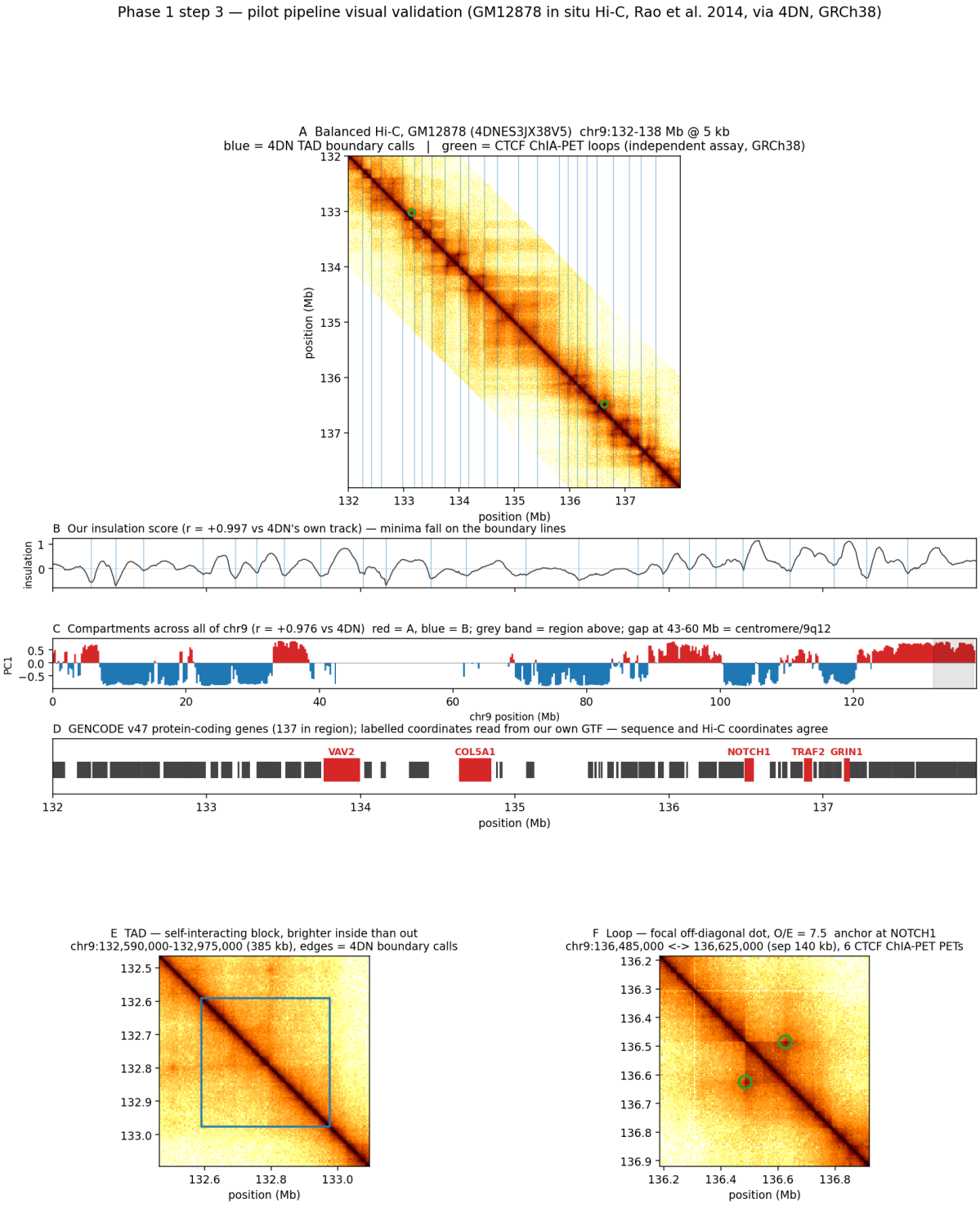

Figure 1: Pilot pipeline validation. (A) Balanced contacts with 4DN boundary calls in blue and CTCF ChIA-PET loops in green. (B) Our insulation score. (C) Compartments across all of chr9; the blank interval at 43–60 Mb is the centromere. (D) GENCODE genes with coordinates read from our own annotation. (E) A 385 kb TAD. (F) The NOTCH1-anchored loop.

6 Failure modes and controls

6.1 Shuffled-structure controls

ID	Control	Destroys	Rules out
S0	zero the signal	everything	establishes the sequence-only floor at identical parameter count
S1	permute across bins	alignment and autocorrelation	primary reliance probe
S2	circular shift, 10 Mb	alignment only	"any smooth auxiliary channel would do"
S3	distance-matched rewire	locus-specific structure	**"structure is genomic distance in disguise"**

Table 6: S3 is the control that can end the project.

Contact probability falls smoothly with genomic distance. If a model performs as well on distance-matched fake structure as on real structure, what it learned is a re-parameterised positional prior and the central claim is false however good the headline number looks.

The gate passes only if degradation under shuffling is positive with a 95% bootstrap confidence interval excluding zero *and* is at least twice the across-seed standard deviation of the real-structure runs. Significance alone is insufficient; the effect-size floor prevents a p-value standing in for an effect.

6.2 The eight failure modes

	Failure	Symptom	Status
F1	bias absorbed into b_δ	trains to baseline loss	highest probability; mitigated by genome-wide standardisation
F2	softplus saturation	structural weights never move	**observed**; b_g changed $-8 \to -4$
F3	structure inferable from sequence	structural input redundant	**largely defused** by Lee's Evo 2 probe
F4	bin longer than memory horizon	trains to baseline loss	**precondition confirmed**; mitigated by interpolation
F5	permeability gate stays at zero	half the mechanism inert	monitor the distribution of p
F6	gradient starvation	structural pathway never learns	1.7% of parameters behind two nonlinearities
F7	forward and reverse cancel	trains to baseline loss	two of eight features flip sign under RC
F8	structural encoder collapses	reduces to F1	**eliminated** if the encoder is removed

F1, F4 and F7 produce the same symptom: the model trains to baseline performance and raises no error. Four diagnostics therefore run continuously—the variance the structural pathway contributes to the timescale, the distribution of the permeability penalty, the growth of the

structural weight norm, and a probe asking whether the baseline model's hidden states already predict the structural signal.

6.3 Memory horizon, measured

At the reference initialisation, horizons run from 0.6 tokens at the fast end to 994 at the slowest, median 14.9. A 5 kb bin is 5,000 tokens, so no channel-state pair spans a single bin; $\tau_{\max}/\text{bin} = 0.199$. Sixteen layers relaying extends reach to roughly 15,900 tokens, still short of a 385 kb TAD.

Δ is learned and can grow, so this is not proof of failure. It constrains the sequence-only baseline exactly as much as the structural model, which raises a question worth settling before spending GPU time: can a model of this size represent TAD-scale dependencies at all? **Phase 3 must therefore report the trained memory horizon, and that measurement gates Phase 4.**

7 Decisions of record

	Decision	Choice	Reasoning
1	Mechanism	structural bias in the recurrence	unreachable by contrastive alignment on output embeddings; contains hard state resets as its limit; cheap to falsify
2	Reverse complement	Caduceus-PH (augmentation)	equivariance is orthogonal to the hypothesis; coupling them makes a null result uninterpretable
3	Hi-C resolution	5 kb, 1 kb as ablation	comparability with CHROME; both levels are in the same file
4	Cross-species scope	out for this paper	Evo2HiC uses 177 species, HiC-Foundation 316; entering that lane splits compute against better-resourced groups
5	Permeability term	**keep**, via custom kernel	retains the soft-reset semantics and the link to future work

8 Change log

Every substantive change to the project is recorded here with its reason.

2026-08-31 (fifth entry) — the multi-chromosome index is finally reachable from training, and three provenance defects are closed

No GPU. `dataset_index_multichrom.npz` was built on 2026-08-26 and **no code had ever read it**: `WindowDataset.__init__` hardcoded `np.load(PROCESSED / "dataset_index.npz")` and the flat `train/val/test/phi/usable` keys. The leakage-safe split — the review's essential item 5 — was therefore not reachable by any training run. Closed:

- `WindowDataset` takes `index=` and **detects** the schema via `"chrom_names" in z.files` rather than guessing from the filename. The multichrom path holds `phi/usable/tokens` as per-

chromosome lists indexed by {split}_chrom_id; each item now also carries chrom_id (−1 on the single-chromosome path).

- **The single-chromosome attributes are left None on the multichrom path, deliberately.** phase5_*.py and test_phase4_wiring.py reach into ds.phi/ds.usable directly; aliasing one chromosome would let them keep running while silently describing one seventh of the data. A TypeError at first use is the correct outcome.
- φ controls are applied **per chromosome**. S1 GLOBAL-PERM is ambiguous across seven chromosomes, and permuting across them would additionally destroy the per-chromosome standardisation the index records. Recorded in run_config.yaml as phi_control_scope.
- train.py gains --index, threaded through make_loader.

Verified: train n=14,600 on chr10–13, val n=2,455 on chr14–15, test n=1,647 on chr9 only — no chromosome in two splits. position/window/dual granularities, S1 and S2, RC augmentation and both baseline guards all exercised on the new path. test_phase4_wiring.py **60/60**, test_model.py **21/21**, both unchanged.

Three provenance defects closed in the same pass. (i) The --window bug: run_config.yaml recorded args.window, which defaults to 32768 and is not passed by run_phase4.sh, so a 65 kb run wrote the wrong window and *half its true token count* into the one artefact whose job is provenance. Both the recorded window and tokens_per_optimizer_step now come from the loaded index; window_arg and window_arg_matches_index are kept beside them so a mismatch is visible rather than silent. (ii) run_config.yaml now records index, index_schema, split_by, chrom_roles and phi_granularity — a run states which dataset it used. (iii) phase4_guard.sh's completion sentinel was a bare ^ALL SEEDS DONE, which a log from a different configuration satisfied, so every 65 kb launch exited immediately while appearing to have run. Both guard and supervisor now scope it to RUN_TAG (default w65536_multichrom); round-trip verified to match its own tag and reject a stale one.

2026-08-31 (fourth entry) — IMR90 FAILS the pre-registered B3 gate on the one feature that motivated choosing it; K562 passes

No GPU. IMR90 chr14 acquired and featurised (--cell-line IMR90, set 4DNES1ZEJNRU). The gate in docs/B3_DEPTH_GATE.md, fixed before any of these numbers existed, returns **FAIL**:

	GM12878	K562	IMR90
usable_frac (gate ≥ 0.70)	0.8039	0.8093	0.7999 (pass)
r insulation_100kb (gate ≥ 0.95)	0.9971	0.9966	**0.9973** (pass)
r compartment (gate ≥ 0.90)	0.9711	0.9796	**0.8815** (FAIL)

The threshold was not moved. The gate document states that partial failure is not rescued by lowering a threshold in it, and it has not been.

Diagnosis, which does not change the outcome. Two checks were run to separate "shallow map" from "our bug". Within quartiles of |PC1| the correlation is 0.9209/0.9857/0.9929/0.9196 and sign agreement is 0.9680 — far above the pooled 0.8896, which indicates a monotone nonlinearity rather than noise. But Spearman only partly closes the gap: IMR90 0.9254 against GM12878 0.9685 and K562 0.9703. IMR90 is worst of the three on the rank statistic too, so the failure is real and not an artefact of choosing Pearson.

The informative asymmetry. IMR90's insulation agreement is 0.9973, the *best* of the three lines, while its compartment agreement is the worst. A map starved by shallow sequencing would degrade both. This is therefore specific to the compartment eigenvector, not to map depth — and compartment_pc1 is exactly the feature whose GM12878-vs-IMR90 divergence ($r = 0.6035$, third entry) was the reason for preferring IMR90. The failure lands on the feature that motivated the choice.

Where this leaves B3, PI's call. K562 is now the only line that passes, and it passes cleanly — but it carries the CNV confound argued in the third entry, which is a real objection and not withdrawn. The options are: use K562 with the aneuploidy documented as a limitation; drop the second cell line and scope the paper to one; or investigate IMR90's compartment pipeline further, on the explicit rule that if no computational fault is found, IMR90 stays out by the gate as written. Extending to chr15 would be new information for that decision, *not* a re-run of the gate until it passes.

Nothing is trained; no cross-cell-line claim is licensed either way.

2026-08-31 (third entry) — B3 second cell line: the depth caveat does not bind, and IMR90 beats K562 on the quantity that matters

No GPU. K562 chr14 acquired end to end (`--cell-line K562`); the 7.9 GB mcool was streamed over range reads, never downloaded. Band assembled: 2,912,263 unique upper-triangle entries in 230.2 s.

The depth gate was pre-registered before the numbers existed (`docs/B3_DEPTH_GATE.md`, written while the band was still streaming) because `phase1_acquire.py` warns that GM12878's set is 72 experiments merged against K562's 6, and asks for confirmation that the shallower map does not starve the balancing weights. **It does not.** K562 chr14 passes all three criteria, and its usable-bin fraction is marginally *higher* than GM12878's on the same chromosome:

	GM12878	K562
usable_frac (gate ≥ 0.70)	0.8039	**0.8093**
r insulation_100kb vs own 4DN track (gate ≥ 0.95)	0.9971	0.9966
r compartment vs own 4DN track (gate ≥ 0.90)	0.9711	0.9796

So depth does not decide which second line to use, and the choice falls to which line is more informative. Measured on chr14 at 5 kb from the three lines' own 4DN processed tracks (`pybigtools`, Pearson on jointly finite bins):

pair	insulation r	compartment r
GM12878 vs K562	+0.7513 (n=17,306)	+0.8250 (n=17,450)
GM12878 vs IMR90	+0.7629 (n=17,209)	**+0.6035** (n=17,350)
K562 vs IMR90	+0.7903 (n=17,279)	+0.6062 (n=17,350)

IMR90 is markedly more divergent from GM12878 in A/B compartments (0.6035 vs 0.8250) while insulation is a near tie (0.7629 vs 0.7513). **The insulation column is an internal control**: were the compartment gap driven by pipeline or depth differences between experiment sets, insulation should degrade alongside it, and it does not. The gap therefore reads as biological, and matches lineage — GM12878 (lymphoblastoid) and K562 (erythroleukemia) are both haematopoietic, whereas IMR90 is lung fibroblast. `compartment_pc1` is one of the eight φ features, and it is the one carrying cell-type identity.

Recommendation, PI's call: IMR90 as the B3 second line. Beyond the divergence measured above, K562 is near-triploid with extensive copy-number variation; `log_contact_density` is a φ feature and compartment calling is CNV-sensitive, so part of any GM12878-vs-K562 φ difference would be karyotype rather than regulatory architecture — the same class of benign explanation S4 exists to exclude. K562 also mismatches the GRCh38 reference the model reads. IMR90 is karyotypically normal diploid and its set has 7 experiments against K562's 6.

Caveats. One chromosome, one resolution, 4DN-processed tracks; the insulation control argues against a purely technical reading but does not exclude it. The K562 chr14 build is retained — as a *same-lineage* second line it is a useful gradient against IMR90's different-lineage contrast, should the effect ever need to be shown to scale with structural divergence. Nothing here licenses a cross-cell-line claim: no model has been trained.

2026-08-31 — the permeability term p was missing from every τ ever reported: the structural arm's memory horizon is ∼51 tokens, not ∼487, and F4's fix never reached it

No GPU. Found while verifying an external peer review, which alleged a sign error in the τ equation. **The alleged sign error does not exist**: `architecture_spec.md` line 173 reads $\tau = -1/(\Delta A - p)$ with $A < 0$, which is algebraically identical to the reviewer's proposed correction $1/(\Delta|A|+p)$; verified numerically, both give 55.09338328466751 at $\Delta = 10^{-6}$, $|A| = 1$, $p = 0.018150$. The reviewer dropped the leading unary minus.

Checking it surfaced a real and larger defect. **Both** τ estimators omit p entirely. `BiMambaLM.tau_stats()` computed `tau = 1.0 / (delta * A)`; `train.py::empirical_tau` computed `tau = 1.0 / (delta * absA)`. `empirical_tau` hooks `W_dstruct` explicitly and carries a long docstring warning that reading τ from `dt_proj` alone would be a silent error — it guarded that trap and fell into the adjacent one. The omission is documented nowhere: the bias-only/empirical distinction in `phase3_report.py` is about the input-dependent Δ term only, and F4's analysis in `architecture_spec.md` §4.1.4 never mentions p.

Why it matters. p is constructed only in the structural branch (`model.py`: `if self.c.structural and s is not None`), so the baseline has no p term at all and its reported τ is correct. The structural arm carries $p = \text{softplus}(b_g)$ at every position, and p clamps exactly the quantity `dt_floor` clamps — τ can never exceed $1/p$ however small `dt_min` becomes. This is the F4 trap a second time, in a second constant.

Recomputed from the three logged Phase 4 checkpoints (`results/novel_model/w32768/structural_seed{0,1,`
state dict loaded with `missing=0 unexpected=0`):

seed	trained b_g range	τ median as reported	τ median with p
0	$[-4.1452, -3.9839]$	479.8	50.56
1	$[-4.1107, -4.0066]$	487.6	51.24
2	$[-4.1303, -3.9635]$	452.8	51.13

The recorded claim "every structural τ median exceeds every baseline τ median (+52.6 tokens); memory lengthened" is therefore **inverted**. Against the baseline's 434.7, the structural arm's true median is ∼51 — roughly 8.5× *shorter*. Retention of the slowest channel across one 32,768 bp window: baseline $\exp(-10^{-6} \cdot 32768) = 0.9678$; structural, at the D2-measured $p = 0.017234$, $\exp(-(10^{-6} + 0.017234) \cdot 32768) = 5.4 \times 10^{-246}$.

This supplies a mechanistic account of the Phase 4 result that no diagnostic had reached. D2 recorded that the gate "never engaged" because p never moved from its initialisation. That was read as p being inert. It is not inert: it was never negligible. $b_g = -4$ was chosen (failure mode F2) to trade init-equivalence against gradient scale, and the note reasoned that $p = 0.018$ is "negligible" — negligible per step, but it is applied at every one of 32,768 positions, and it is precisely the term F4's `dt_min` $= 10^{-6}$ was introduced to make small. The structural arm was handicapped relative to the baseline from step 0 by a constant neither arm was supposed to differ by, which is consistent with the sign of the measured result (+0.0020 bits *worse*).

Fixed in code, no result overwritten. `tau_stats()` now emits p-inclusive `tau_median/p99/max` alongside `*_pfree` fields preserving continuity with everything logged before today, plus `p_bias` and `tau_ceiling_from_p`. New regression test `test_tau_includes_permeability` asserts the $1/p$ ceiling holds, that raising b_g shortens τ ($-4 \to 55.1$ tokens; $-1 \to 3.2$), and that the baseline reports no p term. `scripts/test_model.py` is **21/21** (was 17/17). `train.py::empirical_tau` is patched too: `DeltaCapture` now hooks `w_gate` and exposes `p = softplus(w_gate(s))`, which enters the rate as $\Delta|A| + p$. Verified on untrained models at $l = 256$ — structural τ median 49.14, $\tau_{\max}$ 55.09 (respecting the $1/p$ ceiling of 55.1); baseline τ median 473.85, $\tau_{\max}$ 1,233,002.62.

Two things this does NOT establish, and neither may be asserted without a run. It does not show the Phase 4 null would reverse with p removed; the logit perturbation from p

alone at initialisation is 3.686×10^{-2} against an activation scale of 2.635×10^{2} (ratio 1.4×10^{-4}), measured on one model with all other weights held fixed, at $l = 256$. And whether b_g should change is a `architecture_spec.md` §7 decision-of-record, so it is the PI's, not mine. It is filed as an open decision, not applied.

2026-08-31 (second entry) — provenance manifest, and the per-chromosome standardisation scope stated plainly

No GPU. Peer-review major comment 3 ("a reader cannot determine which code, index, granularity and checkpoint produced each reported result") is correct and is now addressed by `scripts/build_manifest.py`, which regenerates `results/MANIFEST.json` from artefacts on disk rather than hardcoding anything. It records the git commit and dirty flag, and for every index its SHA-256, schema, window and split sizes; for every run its `run_config.yaml` status, window, structural flag, last logged step and checkpoint SHA-256 (first 64 MiB).

First run confirms all eleven logged runs carry `cfg_window=32768` and `status: COMPLETED` at step 2000, except `smoke_seed0` (step 300) and `baselines/baseline_v2_w65536_seed0`, which is `STARTED -- no metrics yet` at step 100 with no checkpoint — a dead 65 kb launch that must not be mistaken for evidence.

On leakage: `dataset_index_multichrom.npz` records `phi_standardisation = 'per-chromosome'`, i.e. each chromosome is z-scored against *its own* usable bins. This is not train-only normalisation — held-out chromosomes are normalised with their own statistics, which is transductive. It is defensible as a deployment assumption (a new chromosome arrives with its own Hi-C map) and was decided deliberately on 2026-08-26, but the global-standardisation sensitivity analysis the review asks for is genuinely owed and is not yet run.

2026-08-30 — T5c and T5c-dual: the per-window and joint conditioning arms, a real architecture change, Decisions 8–9

Requested directly by the PI after a review of the project's own novelty claims concluded the implemented mechanism (Decision 1, per-position Δ-bias) is a narrow, mostly-inert addition to an otherwise-unmodified Caduceus-Ph, on an unchanged pretraining objective. This entry documents the mechanism actually built in response, not the discussion that led to it.

The diagnosed reason Decision 1 is mostly inert (CLAUDE.md §4): keep(φ) = 0.1099 at 65,536 bp means a per-position mechanism can only ever reach ∼11% of φ's variance; ∼89% lies BETWEEN windows and is unreachable by any per-position bias at any window width that fits on this hardware. Decision 8 addresses this directly.

Decision 8 — T5c, per-window conditioning (`phi_granularity="window"`). `scripts/phase1_dataset.py` new `pool_phi_window()` collapses a window's per-position interpolated φ to its mean over valid positions, broadcast to every position. **This needs ZERO changes to** `model.py`: `struct_encoder` and `W_dstruct` (Decision 1's own pathway) are both pointwise – applied independently per position, nothing mixes across positions – so a CONSTANT input produces bit-identical output to computing one structural embedding for the window and broadcasting it. "Encode once, broadcast" and "broadcast the input, encode redundantly at every position" are the same computation. The entire arm is therefore a data-pipeline change: what `WindowDataset` hands the already-verified pathway, not a new pathway. Zero additional parameters; the trained model class is byte-identical to Decision 1's.

Decision 9 — T5c-dual, joint local + global conditioning (`phi_granularity="dual"`). Decision 8 alone TRADES the within-window share for the between-window share; it does not get both. T5c-dual concatenates the per-position and per-window-pooled φ into one 16-channel input (`d_struct_raw` $8 \to 16$), validity defined as the AND of both halves (a position is trusted only where both scales are legitimately defined). Still no new architecture – the same pointwise encoder, now with a wider input – and negligible additional cost: parameter budget

measured at **+0.429%** against Decision 1's +0.43%, i.e. the structural encoder's first layer gaining $8 \times d_{\text{struct\_hidden}}$ weights and nothing else. This is the version that can be credited or faulted for using ALL of φ's variance, not half of it by construction.

Wired end to end, not just unit-tested. `scripts/test_phase4_wiring.py` grew from 51 to **60/60** checks: shape and validity checks for both new granularities, a check that a `d_struct_raw` mismatch fails LOUDLY (shape error) rather than silently, a parameter-budget check for the dual arm, and RC-augmentation checks confirming pooled φ stays constant and its symmetry vector doubles correctly by concatenation (an antisymmetric channel's window mean under reversal-plus-sign-flip is the negative of the original mean, same as any single position, so no special-casing was needed). Beyond the unit tests, a real `scripts/train.py --smoke` run was executed for both new granularities – actual forward/backward/optimiser steps, not an isolated check – confirming `run_config.yaml` records `phi_granularity: dual` and `d_struct_raw: 16` correctly and that training proceeds without error (5 steps, CPU-adjacent tiny config, one GPU, ∼8–9s).

What this is not. Not a new backbone; Caduceus-Ph's architecture is untouched. Not a new pretraining objective; the loss is the same masked-nucleotide task as Decision 1 and as Caduceus itself. It is a second (and third) instantiation of the SAME kind of contribution as Decision 1 – a targeted, cheaply falsifiable addition to an existing model – aimed at the specific, measured limitation Decision 1's own diagnostics found. No training run has been LAUNCHED at either new granularity beyond the `--smoke` wiring check above; Phase D (a real, properly-powered run) is still gated on the seed-budget and endpoint decisions already on record.

2026-08-26 (sixth entry) — Phase B1: multi-chromosome data build complete, chr9 held out whole as test, a real concurrency bug found and fixed

No GPU. `scripts/phase1_acquire.py` and `scripts/phase1_features.py` gained `--chrom`; new script `scripts/phase1_multichrom_index.py` writes `data/processed/dataset_index_multichrom.npz`. Six new chromosomes acquired end to end (Hi-C band + coarse matrix + sequence + GENCODE annotation + φ features, each validated against independent 4DN tracks), –dry-run'd first per the plan's trap warning.

Roles. chr9 held out in its **entirety** as test — no internal train/val/test carve-out inside chr9 the way the single-chromosome `dataset_index.npz` has, matching CHROME's chr9-held-out convention. chr10–chr13 train, chr14–chr15 val. Measured total: train ∼957 Mb, val ∼161 Mb, test ∼108 Mb — close to the ∼1 Gb target (`docs/NEXT_SESSION.md` §4).

φ standardisation decided PER-CHROMOSOME (recorded in `phase1_features.py`'s module docstring): each chromosome's φ is z-scored against its own usable bins only, unchanged from the single-chromosome pipeline. A global standardisation would require every chromosome's raw φ simultaneously and would make one chromosome's z-scores depend on which others were included in a given build. Cost: keep(φ) and every φ-derived number on this index is relative to each window's own chromosome's variance, not one shared absolute scale.

chrom	role	usable bins	insulation$_{100kb}$ r vs 4DN	compartment r vs 4DN
chr9	test	77.74%	0.9969	0.9759
chr10	train	94.84%	0.9969	0.9612
chr11	train	94.11%	0.9972	0.9774
chr12	train	96.37%	0.9971	0.9585
chr13	train	82.58%	0.9968	0.9655
chr14	val	80.39%	0.9971	0.9711
chr15	val	72.64%	0.9969	0.9482

chr13–chr15's lower usable-bin fractions track their acrocentric short arms (large centromeric/repetitive regions), the same phenomenon behind chr9's own 22.26% unusable fraction — a property of those chromosomes, not a pipeline defect.

A real concurrency bug, found and fixed. Running five chromosome fetches in parallel (network-bound, no GPU, ample RAM/CPU headroom measured first: 245 GiB free, 40 cores) exposed that `stream_download`'s destination for the GENCODE GTF is genome-wide, shared across every chromosome, and had no protection against concurrent writers. Three of six chromosomes (chr13, chr14, chr15) crashed racing on that one file: one hit `zlib.error: Error -3 while decompressing data: invalid block type` reading a file a sibling process was simultaneously overwriting; two hit `FileNotFoundError` on `tmp.replace(dest)` because a sibling's rename had already consumed the shared temp file first. `stream_download` **now takes an** `flock` **on a sibling** `.lock` **file before touching any destination**, and treats an already-complete destination (found after acquiring the lock) as a cache hit even with no checksum to verify it against — the process that loses the race for the lock finds the file already there when it wakes. The three affected chromosomes' expensive Hi-C band/coarse/sequence fetches (already correct on disk) were **not** re-run; only the annotation step was recovered from already-verified files, using the fixed locking code.

Not yet done. No chromosome beyond chr9 has had φ visually inspected against a known TAD/loop (`docs/data_card.md` §6 is chr9-only). The quantitative validation above is strong and sufficient to proceed per that section's own stated logic, but the visual sign-off should be extended before this data supports a paper claim. Nothing has trained on this index; Phase D is out of scope for this session.

2026-08-26 (fifth entry) — Component 3 amended, not deleted: the alpha confound is ruled out and kept as a negative control; the superseding stratification test is pre-registered in §7

Two corrections to how the previous entry is read, both from the PI, before Phase B started.

- **The alpha-confound result is a negative control, not support for the dropped claim.** A1 ruled out one specific alternative explanation for $\rho = +0.1210$ — that it was an artifact of fitting each arm's ridge probe at its own LOO-GCV-selected α. ρ stayed in $[+0.112, +0.118]$ under a shared α, and Q1 stayed negative. That is recorded as what it is: one confound eliminated, not evidence for the stratification.
- **The A1 gate was mis-specified, and the mistake is recorded so it is not repeated, not to reopen the chr9 verdict.** The gate paired Spearman ρ with a requirement that a four-bin quartile frac>0 sequence be *exactly* monotone. That second condition was a descriptive display of the same trend as ρ, not an independent test, so failing it added no real information beyond what ρ itself already said. **The chr9 verdict stands**: component 3 is still dropped from the four-part claim on chr9's 8.98 Mb, where the block-length conditions needed to test ρ properly (§7 below) are DISJOINT and cannot both be satisfied. Nothing about chr9 is re-scored.

Superseding pre-registration written into `docs/architecture_spec.md` **§7 (Decision 7) verbatim**, before any multi-chromosome data exists:

> Pre-registered 2026-08-26, superseding the withdrawn chr9 version.
>
> HYPOTHESIS. The structural arm's probe-B advantage concentrates in windows where phi varies within the window.
>
> PRIMARY AND ONLY TEST. Spearman rho(within-window phi variance, d_w) > 0 on held-out chromosomes, moving block bootstrap, blocks ≥ 1 Mb AND ≥ 20 blocks. Both conditions are satisfiable only above ~40 Mb of evaluation genome; on chr9's 8.98 Mb they are mutually exclusive, which is why chr9 could not test this in either direction.
>
> NO SECONDARY. The previous version paired rho with a requirement that the quartile frac>0 sequence be exactly monotone. That was a four-bin descriptive display of the same trend, not an independent test, and it is not repeated.

> PRIOR RESULT, DISCLOSED. On chr9 this failed its conjunctive gate on 2026-08-26: rho was stable at +0.112 to +0.118 under shared-alpha refitting against +0.1210 originally, and Q1 remained negative, but the quartile monotonicity held at only 1 of 3 alpha settings. The alpha confound was ruled out; the trend was not established.
>
> DISCONFIRMING OUTCOME. rho CI includes zero under the stated block conditions.

This test becomes runnable for the first time once the multi-chromosome build (`docs/RESEARCH_PLAN_2026-08-` Phase B1, below) provides enough evaluation genome to satisfy both block conditions simultaneously.

2026-08-26 (fourth entry) — B0(a): the single-GPU allocator smoke test, and a correction to what was blocking 65,536 bp launches

No training. `scripts/b0_single_gpu_smoke.py`, a few forward/backward/ optimizer steps only, output written to `results/b0_single_gpu_smoke_*.json`. This was the untried mitigation flagged in §6 for over a week: does the `NVML_SUCCESS == DriverAPI::get()->nvmlInit_v2_()` assertion at `CUDACachingAllocator.cpp:983` still fire with only one device visible to the process (`CUDA_VISIBLE_DEVICES=0`, no DDP, no gloo)?

Result: it still fires, at 65,536 bp, on one GPU, with no other process using either device (`mem_get_info` showed 43.97/44.39 GiB free before the model was even built). **Single-GPU alone is not the fix.** But two further runs isolate why:

window	grad-checkpoint	peak (GiB)	s/step	outcome
65,536	off	—	—	nvmlInit_v2_ assertion
32,768	off	37.23	∼3.0	OK
65,536	**on**	**7.51**	∼7.16	**OK**

37.23 of 44.39 GiB at 32,768 without checkpointing is already 84% of the device; activation memory in the scan is documented to grow roughly linearly in window length (`phase5_memcheck.py`), so 65,536 without checkpointing would need on the order of 70+ GiB — over the 44.39 GiB card, on ONE GPU, with nothing else running. **This reproduces the exact assertion text previously attributed to "concurrent load from an unrelated project on both devices," with no unrelated project and no second device involved.** The 2026-08-10 entry already recorded that this box's caching allocator disguises an out-of-memory condition as an NVML assertion; every prior 65,536 bp launch attempt combined that pre-existing box quirk with a request that could not have fit in either GPU regardless of contention, because `--grad-checkpoint` defaults off and `run_phase4.sh` never passes it.

The fix was already built and sitting unused. `--grad-checkpoint` (fixed 2026-08-18, CLAUDE.md §6) makes 65,536 bp fit in 7.51 GiB on one GPU at ∼7.16 s/step, measured over 5 steps just now. This is not the official D2 memcheck run — that still needs `phase5_memcheck.py` re-run in full with its output persisted, per the workplan — but it is a real, disk-backed number for this one configuration, and it changes the D3 launch plan: `--grad-checkpoint` **must be passed at 65,536 bp; it is not optional the way the 32,768 bp entry (2026-08-10, second entry) found it to be.** Whether DDP across both GPUs together also needs it, and at what batch size, is not yet measured and is explicitly out of scope for this narrow smoke test.

2026-08-26 (third entry) — Phase A1: the phi-variance stratification is a lead proposed, tested against its own alpha confound, and killed

`docs/RESEARCH_PLAN_2026-08-26.md` written first, committed, so the phase list survives a culler kill. Phase A1 is the first item on it: the stratification result two entries below (Spearman $\rho(\varphi \text{ variance}, d_w) = +0.1210$, fraction-positive monotone across quartiles) was computed with

each of the six runs fitted at its OWN LOO-GCV-selected ridge α. Those six αs are not equal across arms — structural seeds all select 3.455×10^5; baseline seeds 0/1 select 2.031×10^5; baseline seed 2 rails to the grid maximum, 10^9. A probe regularised differently per arm can move "who wins where" for reasons that have nothing to do with structure, so before this result is pre-registered its dependence on that asymmetry has to be measured, not assumed away. No GPU; new script `scripts/phase5_common_alpha.py`, output `results/novel_model/p5_common_alpha.json`.

Method. All six runs refitted at a single SHARED α (no per-run LOO-GCV), swept over the geometric mean of the six selected values, 1.0926×10^6, and one decade either side. Quartile table, frac>0 trend and the Spearman block-bootstrap sweep recomputed identically at each.

α	Q1 mean d_w	frac>0 by quartile	monotone?	ρ	gate
1.093×10^5 (geo/10)	−0.0208	0.377, 0.603, 0.574, 0.507	**no**	+0.1137	fail
1.093×10^6 (geo)	−0.0111	0.464, 0.529, 0.603, 0.594	**no**	+0.1183	fail
1.093×10^7 (geo*10)	−0.0066	0.478, 0.559, 0.588, 0.609	yes	+0.1116	pass

Q1 stays negative at all three αs, and ρ stays in a tight band, +0.112 to +0.118, next to the original per-run-alpha value +0.1210 — the alpha confound does *not* explain ρ away, which is itself informative. **But the fraction-positive trend, pitched as the monotone, assumption-free half of the finding, is monotone at only one of the three shared-alpha settings tested** (it breaks between Q3 and Q4 at the geometric mean itself, 0.603 → 0.594, and between Q2 and Q3 at geo/10, 0.603 → 0.574). The pre-specified gate in `docs/RESEARCH_PLAN_2026-08-26.md` is conjunctive — Q1 negative **and** frac>0 monotone — and the second half does not clear it at 2 of 3 alpha settings. The block-length disjointness reported in the entry below (reliable blocks ≤ 0.262 Mb; Hi-C-valid blocks ≥ 1 Mb have ≤ 9, below `MIN_BLOCKS = 20`) reproduces identically at every α in this sweep and is not resolved by this check either way.

Verdict: A1 FAILS the pre-registered gate. Per `docs/RESEARCH_PLAN_2026-08-26.md` Phase A2's failure branch: this is not pre-registered in §7. **Component 3 of the four-part claim ("keep(φ) made a prediction and it held") is retracted before it was ever written into an architecture-spec decision, and the project proceeds on three components**: the mechanism is novel, the F4 finding generalises, and the null is diagnosed (keep(φ) leaves 89% of φ's variance between windows even at 65,536). The stratification remains recorded as a lead for the multi-chromosome build (Phase B1) to confirm or kill on genome large enough to make blocks ≥ 1 Mb **and** ≥ 20 of them simultaneously satisfiable, which 8.98 Mb of chr9 val cannot do at any α.

2026-08-26 (second entry) — T2 hardened: a ridge boundary hit corrected, a flat sign test added, and keep(φ) turned from an explanation into a tested prediction

Four corrections and one new result, all on the same cached representations. No GPU. Supersedes nothing in the entry above except the exact r of one run.

- **CORRECTED: `baseline_v2_seed2`'s ridge penalty was clamped, not chosen.** `ALPHAS = np.logspace(-3, 6, 40)` and that run selected $\alpha = 10^6$ *exactly* — the grid maximum. LOO-GCV had railed against the boundary, so its regularisation was set by where the grid stopped. It is also the strongest baseline seed, the one doing most to close the gap, and an under-regularised baseline widens the structural-minus-baseline difference *in the direction this project wants*. Grid extended to `logspace(-3, 9, 53)` and `ridge_fit_eval` now returns `alpha_at_grid_edge` rather than letting a boundary hit pass silently.
  Effect on the result: negligible. `baseline_v2_seed2` moves +0.0221 → +0.0222 and still selects the new maximum (10^9), i.e. its LOO optimum is effectively unbounded —

as $\alpha \to \infty$ the ridge predictor converges to a fixed direction and r, being scale-invariant, converges with it. The arm difference is +0.01032 against +0.01033 before. The flag was right, the fix was necessary, and the conclusion does not move.

- **ADDED: the sign test, and it is flat.** 141/274 windows favour the structural arm — 51.5%, exact binomial $p = \mathbf{0.6725}$. This is the most assumption-free statement available: no ridge, no bootstrap, no block length, no distributional assumption. It corroborates the honest reading ($p \approx 0.09$) and contradicts the 9-block artefact ($p = 0.0068$), which is exactly why it belongs in the report. The summary now also states that $\mathrm{sd}(d_w) = 0.0837$ is **ten times** the mean of +0.00835.
- **CORRECTED: the ACF does not decay cleanly.** With $n = 274$ the band is $2/\sqrt{274} = 0.1208$. Lags **1, 8 and 13** exceed it (+0.171, +0.171, and one more), i.e. 3 of 60 — about what noise alone gives. Reporting "first below the band at lag 2" described the first crossing and implied a decay that is not there. Now reported as the full list of exceedances.
- **STATED: why the 1 Mb floor overrides the measurement.** The measured ACF says dependence dies within 0.066 Mb, and the block length was floored at 1 Mb anyway. That looked like an unexplained override. The reason: d_w has sd 0.0837 against a mean of 0.00835, so it is mostly noise, and **the autocorrelation of a mostly-noise statistic is near zero whatever the underlying data do**. Hi-C dependence is a megabase property of the data. Letting the short measured lag pick the block size would be letting the noise floor pick it, in the anticonservative direction.
- **NEW RESULT — keep(φ) makes a prediction, and it holds.** Until now keep(φ) = 0.0573 explained an absence, which is the weakest kind of claim. It predicts something testable: if it is the true account, the structural advantage should concentrate in windows where φ actually varies *inside* the window. Per-window within-window φ variance spans 0.0028 to 13.33 of the pooled val variance — nearly four orders of magnitude — so the strata are real.

quartile	φ variance range	n	mean d_w	fraction > 0
Q1	0.0028 – 0.0659	69	−0.01620	0.420
Q2	0.0673 – 0.2969	68	+0.02168	0.500
Q3	0.2987 – 1.2114	68	+0.01713	0.559
Q4	1.2228 – 13.3271	69	+0.01112	0.580

Spearman $\rho(\varphi \text{ variance}, d_w) = \mathbf{+0.1210}$. The naive `spearmanr` $p = 0.0454$ is **not used** — it assumes independent observations and these are a contiguous series. Block bootstrapped instead, and swept, with the same CI-width diagnostic applied:

L (win)	blocks	p	95% CI	width
1	274	0.0365	[+0.0062, +0.2342]	0.2280
4	69	0.0255	[+0.0132, +0.2319]	0.2187
8	35	0.0230	[+0.0190, +0.2192]	0.2001
15	19	0.0200	[+0.0182, +0.2203]	0.2021
31	9	0.0180	[+0.0257, +0.2276]	0.2019
61	5	0.0005	[+0.0523, +0.2306]	0.1783

The width narrows with L again, so only the $\geq$ 20-block rows are usable. At the largest reliable setting ($L = 8$, 35 blocks) $p = 0.0230$, CI [+0.0190, +0.2192]. **Unlike the pooled-r difference, ρ's CI excludes zero at every block length tested, including the reliable ones**, so this does not rest on the 9-block artefact.

Read it carefully. Mean d_w is *not* monotone — Q2 is the largest, not Q4 — so this is not a clean dose-response. What *is* monotone is the fraction of windows the structural arm wins: $0.420 \to 0.500 \to 0.559 \to 0.580$. And Q1 is **negative**: where φ barely varies within the window, the structural arm is worse. Caveat on that: in Q1 the centred target is close to pure noise, so a per-window r there estimates almost nothing and its sign is weakly determined.
Status: a strong lead, not an established result. The 0.262 Mb blocks that make the bootstrap reliable are shorter than Hi-C autocorrelation, so $p = 0.0230$ is anticonservative, and 8.98 Mb cannot do better. The multi-chromosome build should confirm or kill it. If it holds, it converts keep(φ) from a post-hoc account of a null into a prediction that was made and tested — a materially better paper than either the null or the probe-B positive alone.

- **Correction to the record.** T2 was justified on the expectation that moving the unit of analysis from the seed to the window would buy power "at zero extra compute". That was wrong, and the entry above already says so: 274 windows over 8.98 Mb at megabase dependence is single-digit effective samples, and the window count was never the binding quantity. T2's value is diagnostic, not inferential.

2026-08-26 — T2: re-powering probe B does not reach $p < 0.05$, and the reason is the amount of independent genome, not the test

No GPU, no training. The cached per-position representations from `phase5_positional_probe.py` were reused; no model forward pass was run. New script `scripts/phase5_repower.py`, output `results/novel_model/p5_repower.json`.

- **The P5 globs were broken, and unsafely so.** Every `phase5_*.py` script globbed `NOVEL.glob("structural_seed*")` and `BASE.glob("baseline_v2_seed*")` at the repository root. The 32 kb runs moved to `results/{novel_model,baselines}/w32768/` on 2026-08-18, so those globs matched **nothing** — and a probe that finds no models still prints its composition floor and writes a JSON file. Worse, an *unfinished* 65,536 bp structural run sits at `results/novel_model/structural_seed0` (`status: STARTED -- no metrics yet`, `n_train_windows: 2713`, i.e. the 65 kb index) and the top-level glob matches its name exactly, so a re-run would have mixed one partial 65 kb run into a 32 kb analysis and labelled it a Phase 4 seed. Replaced by `find_runs()` in `phase5_structure_probe.py`, which names the width explicitly and **raises** rather than returning empty.
- **Reconstruction verified against the existing record.** Probe-B pooled r per run, recomputed here from the cache against the *archived* `dataset_index_w32768.npz` (the live index is now the 65 kb build and pairing it with 32 kb representations would be silent nonsense):

structural	r	baseline_v2	r
seed0	+0.0254	seed0	+0.0134
seed1	+0.0303	seed1	+0.0146
seed2	+0.0255	seed2	+0.0221

  These reproduce `CLAUDE.md` §4 exactly. Every structural seed beats every baseline seed; the arm difference is +0.01033.
- **The val set is one contiguous 8.98 Mb stretch** (chr9:120,000,000–128,978,432, 274 windows at a uniform 32,768 bp stride). It is a series, not a sample, so i.i.d. resampling of windows is invalid — it would treat neighbours as independent evidence. A moving block bootstrap was used, 10,000 resamples, seed 20260826.

- **Two window-level estimators, reported both ways.** The mean of per-window r is a weak estimator (each computed on 64 rows: mean +0.00773, sd 0.08077, only 140/274 windows positive). Bootstrapping the *pooled* r difference — the statistic the probe actually reports — is better powered, and is made cheap by the fact that a Pearson r over any union of windows is a function of six per-window sums that are additive.
- **The decisive negative: the bootstrap runs out of blocks before it runs out of dependence.**

L (win)	L (Mb)	blocks	p	95% CI width
1	0.033	274	0.0300	0.01915
4	0.131	69	0.0322	0.01917
8	0.262	35	0.0184	0.01719
15	0.492	19	0.0152	0.01651
31	1.016	9	0.0042	0.01465
45	1.475	7	0.0034	0.01409
61	1.999	5	0.0024	0.01468

  The confidence interval gets narrower as the block gets longer. That is backwards: a longer block retains more dependence and must cost precision, never buy it. It is the signature of a bootstrap built from a handful of draws. The small p-values below $L = 15$ are an artefact of the block count, not evidence, and are recorded here only so that nobody later quotes $p = 0.0042$ from the JSON.
- **What can be claimed.** Nothing at $p < 0.05$. *Across seeds*: exact two-sided permutation $p = 0.100$, which is the attainable floor — 3 vs 3 has 20 arrangements. *Across windows*: at the longest block length with at least 20 blocks ($L = 8$, 0.262 Mb, 35 blocks) $p = 0.0184$, but 0.262 Mb is *shorter* than Hi-C autocorrelation so that figure is anticonservative; at the ≥ 1 Mb blocks Hi-C actually requires, only 5–9 blocks exist and the percentiles are not valid at all. The honest statement is that the truth lies between an anticonservative 0.018 and no valid estimate.
- **Consequence for the plan.** T2's premise — that changing the unit of analysis from the seed to the window buys power — **fails**, and it fails for a reason that no resampling scheme can repair: 8.98 Mb of validation genome is single-digit effective samples at megabase dependence. The binding constraint is the amount of independent genome. That makes **T4 (multi-chromosome) a prerequisite for any significance claim**, not merely the fix for known weakness 5 — the two are the same problem seen from two sides. Raising the seed count would address the other test, but it is the one that needs a GPU.

2026-08-25 (second entry) — work plan written: the two blockers on a positive result are statistical power and φ resolution, and neither needs a GPU

No training ran and no new measurement was taken. This entry records an analysis of the existing record and the task ordering that follows from it, written to `docs/WORKPLAN_2026-08-25.md` and cross-referenced from `CLAUDE.md` §1/§11 and from the head of `docs/NEXT_SESSION.md`.

- **The design cannot express a positive result.** At 3 versus 3 the exact two-sided permutation test has $\binom{6}{3} = 20$ arrangements, so the minimum attainable p is $2/20 = 0.100$. This was already recorded for the P1 gate, but its consequence was not: *every* GPU-hour spent at $n = 3$ buys an experiment pre-committed to non-significance, whatever the mechanism does. Minimum designs that permit $p < 0.05$: **6 paired seeds** ($2/2^6 = 0.031$) or **5 per arm unpaired** ($2/\binom{10}{5} = 0.008$). Recorded with a trap that is easy to get backwards: pairing reduces variance but *shrinks* the permutation space, so at $n = 3$ a

paired sign-flip test ($2/2^3 = 0.250$) is *worse* than the unpaired test already run. Pairing buys power only from $n = 6$ up.

- **A route to significance that costs no compute.** The seed need not be the unit of analysis. `p5_positional_probe.json` covers 274 val windows at 512 bp spacing (17,536 val rows). A per-window arm difference tested with a *block* bootstrap — blocks sized to Hi-C autocorrelation, which runs over megabases — gives n in the hundreds on checkpoints that already exist. Both tests must be reported: they answer "does this reproduce across training runs" and "does this hold across the genome" respectively, and are not substitutes. Recorded with the warning that i.i.d. resampling of windows is invalid here for exactly the reason known weakness 5 is a problem.
- **φ resolution is an untried lever, and it is independent of window width.** `resolution_bp: 5000` is a configuration choice, not a property of the data: `data/pilot_manifest.json` records the source as a multi-resolution `.mcool` and `phase1_acquire.py`:159 opens it as `cooler.Cooler(h5[f"resolutions/{RESOLUTION}"])`. keep(φ) is low partly because φ is a step function on 5 kb bins spanned 6.6 times (32 kb) or 13.1 times (65 kb) per window. Finer bins raise within-window structural variance *without* touching window size, memory or step time. The two levers are independent and only one has been tried. Three module globals carry the resolution: `phase1_acquire.py`:63, `phase1_features.py`:41, `phase1_dataset.py`:38; outputs are already templated on `RES`, so builds at two resolutions coexist without collision and must not be pooled.
- **Staged, with a gate, because the risks are real and unmeasured.** Balanced weights degrade at fine resolution and `MIN_STRUCT_FRAC` = 0.50 may drop a large share of windows; the band scales as $n_{\text{bins}} \times$ band bins, i.e. $(5000/1000)^2 = 25\times$ at 1 kb — **a projection from the measured 7,108,483 entries, not a measurement**. The plan therefore fetches one 10 Mb tile first and gates on three measured numbers (usable-bin fraction against 77.7%, keep(φ) against 0.0490/0.1099, and wall time) before any full-chromosome fetch. A negative outcome at the gate is itself publishable: it would show the constraint is intrinsic to Hi-C's spatial autocorrelation rather than an artefact of binning.
- **CORRECTION to an earlier reading of D2.** The permeability gate bias `b_g = -4.0` (`model.py`:160) was described in discussion as an oversight. It is not: the comment at `model.py`:154–159 records it as a deliberate tradeoff, since the gradient through softplus is sigmoid(b_g) and $b_g = -8$ attenuates learning $\sim$3000×. The correct statement is that **the tradeoff was measured and it lost** — 53× more gradient than $b_g = -8$ still left p_t immobile across 287,309,824 evaluations. Any change here trades away the step-0 identity with the baseline and is therefore a §7 decision, not an implementation detail.
- **Endpoint.** Val bits/nt on masked nucleotide prediction is dominated by local k-mer statistics and is the metric least likely to move under structural conditioning. The plan proposes pre-registering the probe metrics as primary and loss as a secondary "no cost to sequence modelling" result — **recorded before the runs, in `architecture_spec.md`, not after seeing numbers**. This is a §4.1.3 change and is listed as a PI decision.
- **Recorded as a standing constraint: the baseline is not to be weakened.** It is matched-parameter, matched-compute and now paired-init, and that is precisely what would make any separation worth reporting. The plan adds a *stronger* arm instead — a tuned one-hot/GC model — since probe B already places both current arms below the GC composition floor (+0.0802).

Seven items are listed in the plan as requiring a PI decision before implementation. None has been implemented.

2026-08-25 — `CLAUDE.md` brought in line with the 2026-08-17 retractions and the 2026-08-18 scope correction; two provenance defects recorded

No training ran, no model was touched, and no new measurement was taken. This entry records a documentation correction and two defects found while auditing the repository against this log.

- **`CLAUDE.md` was two change log entries behind and stated retracted conclusions as fact.** It was last updated 2026-08-16 and is the file every working session reads first. It carried, as current: "every live direction, in every seed, is in layers 12–15" and "functionally a late readout correction"; "what they produce is near-constant across positions — F1 confirmed by measurement"; and, as the recommended next experiment, "one re-run, no encoder, `d_struct=8`, ∼6 GPU-h". All three were withdrawn by the 2026-08-17 entry above. A session starting from that file would have re-derived a retracted conclusion, or spent ∼9 GPU-h on an ablation this record had already withdrawn.
- **What was changed.** Both retractions are now reproduced in `CLAUDE.md` §4 as marked block quotes with the decomposition table that justifies each, rather than deleted — the retracted text remains visible so a reader carrying the old version recognises it. The `d_struct=8` ablation is marked withdrawn in three places (§4, §6, §12). Added: the keep(φ) = 0.0573 binding constraint, the window scan with its between-window column, the probe B arm separation with both of its caveats attached, and the two-index table. §6 was rewritten from "what to do with a weak pass" to "what to run at 65,536". A banner at the head of the file names both retracted claims so a stale summary is recognisable on sight. Numbers transcribed into `CLAUDE.md` were re-derived from `d1_diagnostic.json` and `p5_vars_diagnostic.json` rather than copied from prose; the D1 band medians and keep ratios reproduce this log's tables exactly.
- **DEFECT, found not fixed: `train.py` records the wrong window and half the token count at 65,536.** `--window` defaults to 32768 (`train.py`:959) and `run_phase4.sh` never passes it. `args.window` is read at exactly two places — `train.py`:769, the `run_config.yaml` dump, and `train.py`:718, `windows_per_optimizer_step` — while the true window length reaches the model from the index. Training is therefore correct and *the record of it is not.* The aborted 2026-08-17 launch already demonstrates this: `results/novel_model/structural_seed0/run_co` records `window: 32768` beside `n_train_windows: 2713`, which is the 65 kb index. Since matching between arms is on steps *and tokens*, this corrupts the one artefact whose purpose is provenance. Fix before launch, by passing `--window 65536` or changing the default.
- **OPERATIONAL, recorded in `CLAUDE.md` §9: the `phase4_guard.sh` false-completion fault will recur.** Noted in the 2026-08-18 entry above and cleared for the current width by archiving the logs; the guard still has no notion of which configuration a console log belongs to, so the next width change reproduces it.
- **Unpersisted measurement, now flagged at the point of use.** The 2026-08-18 entry recorded that `phase5_memcheck.py` output was never written to disk. `CLAUDE.md` §4 now says so directly under the throughput figures, where a schedule would be built from them.

2026-08-18 (second entry) — the 65 kb run launched, was culled at step 100, and the working tree was re-nested by a file-browser drag

- **Measured step time at 65,536 bp: 12.69 s/step** (2×L40S, gloo DDP, batch 2, grad-accum 2, gradient checkpointing on, baseline arm). The 32,768 baseline was 5.30 s/step, so the observed ratio is **2.39**× for a 2× window. Cost is therefore *super*-linear in window length, not linear: the projection used while scheduling this run assumed 11.0 s/step and was optimistic by 15%. One arm of 1,000 steps is ∼3.5 h wall, ∼7 GPU-h; both arms ∼7 h wall, ∼14 GPU-h.
- **Warm start verified in the run's own log:** 274 tensors loaded from `baseline_v2_seed0/checkpoint.pt`

at source step 2,000, 0 left at init. `run_config.yaml` records `init: warm start -- staged pretraining, NOT paired`, as designed — the 32 kb checkpoints predate the paired-init fix and the file says so rather than letting a later reader assume the arms were paired.

- **First attempt was killed at step ∼100 of 1,000** (`Terminated`, i.e. SIGTERM, ∼25 minutes in). `--ckpt-every 120` means no checkpoint had been written, so the attempt was lost entirely and restarted from the warm start. This is the idle culler behaving exactly as §3 describes. **At 12.69 s/step a 120-step checkpoint interval is ∼25 minutes, which is longer than the culler's window** — the interval was chosen at 5.30 s/step, where it was ∼11 minutes. It has not been changed mid-sweep (§12), but it is now mis-sized for this width and the next run at this window should set it lower.
- **INCIDENT: the working tree was re-nested at 11:16:01 UTC.** `results/` was moved inside `src/`, a partial copy of `src/` was placed inside `results/`, and `project-docs/` was moved inside `presentations/`. Every affected directory carries an mtime inside a one-second window, which is a file-browser drag, not a script. Consequences: three partial copies of the results tree existed simultaneously, and `project-docs/` — which §2 rule 4 requires to hold exactly `project.tex` and `project.pdf` — did not exist at its own path.
  Repaired by moving, never deleting. The two partial copies are quarantined under `_misplaced_20260818/` (1.2 GB) pending inspection; the complete tree was restored from `src/results/`. After repair `git status` showed only the quarantine directory and the new 65 kb run directory as untracked, i.e. every tracked path was back where it belongs.
  What this exposed: the trained checkpoints are not in version control. `git ls-tree` finds exactly one `checkpoint.pt` under `results/` (`smoke_seed0`). The six 88–89 MB Phase 3/4 checkpoints exist on disk and nowhere else, and reproducing them costs ∼40 GPU-hours. They survived this incident by luck — the drag happened to move them rather than overwrite them. Nothing in the repository currently protects them from a second one.

2026-08-18 — SCOPE CORRECTION: widening the window is a partial mitigation, not a fix; three defects fixed before launch; staged pretraining proposed

No training ran. This entry records a correction to how the 2026-08-17 result is being described, three code defects found while preparing the launch, and two proposals that are **not** yet decisions.

- **SCOPE CORRECTION:** keep(φ) **is an upper bound, and widening the window does not remove the constraint it identifies.** The 2026-08-17 entry established keep(φ) = 0.0573 at 32,768 bp as a sufficient account of the Phase 4 null. That stands. What must not be written on top of it is "the window was too small and widening fixes it." From `p5_window_scan.json`:

window (bp)	bins	keep(φ)	between-window
8,192	1.6	0.0092	99.1%
16,384	3.3	0.0318	96.8%
32,768	6.6	0.0490	95.1%
65,536	13.1	0.1099	89.0%
131,072	26.2	0.2117	78.8%
262,144	52.4	0.3499	65.0%
524,288	104.9	0.4932	50.7%

  At the adopted 65,536 bp window, **89.0% of φ's variance is still between windows**. At 131,072 it is 78.8%. Even at 524,288 bp — 16× the original window, far beyond what fits in memory — half the variance is still between-window. Widening moves the mechanism

from "almost nothing to act on" to "a little to act on." Any claim that the window was the fault, stated without this table beside it, overstates what was measured.

- **Consequence, and a PROPOSAL not a decision: per-window conditioning.** If ∼90% of the structural variance is between windows at every width that fits in memory, then a *per-position* Δ bias addresses the smaller share *by construction*, and does so at every window width reachable on this hardware. The complementary arm — one structural embedding of the window's φ, injected once per window rather than per position — has access to the between-window share the current mechanism cannot reach. This is an addition to the arm set and therefore a §4.1.3 change: **PI decision, not adopted here.** It is recorded now because it follows directly from a measurement already in the record, and because it is cheap relative to the runs currently scheduled.
- **Window widened, index rebuilt.** `WINDOW` 32,768 → 65,536, `STRIDE_TRAIN` 32,768 (50% overlap), `STRIDE_EVAL` 65,536. Rebuilt index: **2,713 train / 137 val / 122 test**, 13.1 bins of φ per window. The 32,768 index is preserved as `data/processed/dataset_index_w32768.npz` and the 32,768 checkpoints and console logs under `results/{novel_model,baselines}/w32768/`. The 32 kb and 65 kb runs are **different datasets and must not be pooled**, for the same reason v1 and v2 baselines must not be.
- **DEFECT: `--grad-checkpoint` had never worked.** `train.py` called `torch.utils.checkpoint.checkpoint` without importing that submodule — `import torch` does not bind it — so the flag raised `AttributeError` on first use. It defaults to off, so no run had ever exercised it and the fault was invisible for the whole project. Fixed by an explicit `import torch.utils.checkpoint` (`train.py`:73). `run_phase4.sh`:93 now passes `--grad-checkpoint`; the 65 kb window is the first configuration that needs it.
- **DEFECT FIXED: the arms now share an initialisation** (known weakness 3). The structural model instantiates extra modules, consuming RNG, so `structural_seedN` and `baseline_v2_seedN` did not share an init despite sharing a seed label. `train.py` now constructs a reference `BiMambaLM(ModelConfig(structural=False))` under the same seed and copies its parameters into every shape-matching tensor of the structural model. Verified **275/275 tensors bitwise identical**, with $W_{\Delta s}$ still exactly zero. An earlier attempt — reseeding per parameter from a name hash — was **caught before launch and discarded**: it derived each std from the already-constructed tensor, which is itself arm-dependent, and used `hash()`, which Python salts per process, so two arms in separate processes would have diverged regardless. It verified at 90/275.
- **Checkpoint retention.** New `--keep-every` (default 1,000): retained checkpoints at fixed step intervals, so any later diagnostic gets a trajectory rather than a single endpoint.
- **OPERATIONAL: `phase4_guard.sh` reports false completion across configurations.** Line 33 greps `results/novel_model/train_console.log` for `^ALL SEEDS DONE`. The 32 kb log satisfied that string, so the guard exited immediately on every 65 kb launch while appearing to have run. Archiving the logs to `w32768/` cleared it. The guard has no notion of which configuration a log belongs to; this will recur at the next width change.
- **Launch is blocked by GPU contention, not by this repository.** Every 65 kb launch attempt failed with `NVML_SUCCESS == DriverAPI::get()->nvmlInit_v2_() INTERNAL ASSERT FAILED at CUDACachingAllocator.cpp:983`, under concurrent load from an unrelated project on both devices. Consistent with §3: NVML is broken on this box, and the allocator's NVML path is reached under memory pressure. Falling back to single-GPU is the untried mitigation.
- **PROPOSAL, not a decision: staged pretraining (warm start).** The 32 kb checkpoints are 2,000 steps of trained DNA representation, and a Mamba SSM has no positional embeddings, so those weights load into a 65,536 window unchanged. Continuing from them rather than restarting would address known weakness 1 (not converged — val loss still de-

scending at step 2,000) and the window question with one budget instead of two. The cost is that the 32 kb checkpoints were trained *before* the paired-init fix above, so warm-starting carries weakness 3 forward unrepaired. Fresh training at 65,536 is the only route that gets both. This is a PI decision and is recorded unresolved.

- **Outstanding measurement.** Forward+backward memory and step time at 32,768 / 65,536 / 131,072 were measured on 2026-08-17 with `scripts/phase5_memcheck.py`, but the console output was not persisted and no numbers from it are quoted here or anywhere else in this document. Any schedule must not be built on them until that run is repeated with its output written to disk.

2026-08-17 — RETRACTION: the D1 depth profile is a denominator artifact, and the binding constraint is measured to be the window, not the mechanism

Three analyses were run on the finished Phase 4 checkpoints. Inference only; no training, no checkpoint written. One of them retracts a conclusion recorded in the 2026-08-16 (third entry) above, so that is stated first and in full.

- **RETRACTED: "the pathway is inert through the encoder and live only next to the output head."** D1 is a ratio, $\mathrm{Var}_t(W_{\Delta s}s) \,/\, \mathrm{Var}_t(\mathrm{dt\_proj}\,\delta')$. Decomposing the stored per-direction numerator and denominator in `d1_diagnostic.json` by depth band gives medians:

seed	band	$\mathrm{Var}_t(W_{\Delta s}s)$	$\mathrm{Var}_t(\mathrm{dt})$	D1
0	L00–L11	1.1060e-06	2.2285e-02	0.0001
0	L12–L15	2.7630e-07	1.4347e-06	0.1671
1	L00–L11	3.8980e-07	2.5165e-02	0.0000
1	L12–L15	1.8843e-07	3.4627e-03	0.0002
2	L00–L11	1.6919e-07	2.5406e-02	0.0000
2	L12–L15	6.6493e-08	2.3558e-06	0.0280

  Late/early ratios: the structural numerator **falls** (×0.25, ×0.48, ×0.39) while the sequence-driven denominator **collapses** (×6e-5, ×0.1376, ×9e-5). D1 rises in layers 12–15 because what it divides by falls away, not because the structural pathway becomes more active there — in absolute terms that pathway is 2–4× *weaker* in the "live" band than in the "dead" one. The claim that the mechanism degenerated into a late readout correction, which was this project's sharpest self-criticism, is **not supported by its own stored measurements**.
- **RETRACTED: "what they produce is near-constant across positions — F1 confirmed by measurement."** A scale-free measure of positional variation, $\mathrm{keep}(x) = \overline{\mathrm{Var}_t(x)}/\mathrm{Var}_{\mathrm{global}}(x)$, was computed per direction on all 274 val windows (`scripts/phase5_vars_diagnostic` `p5_vars_diagnostic.json`). Medians for $W_{\Delta s}s$, L00–L11 vs L12–L15: 0.1945 vs 0.1952 (seed 0), 0.1195 vs 0.1169 (seed 1), 0.0794 vs 0.0752 (seed 2). The early layers vary with position *indistinguishably as much*, proportionally, as the live ones. They are producing a small position-varying signal, not a bias.
- **The encoder is exonerated; the `d_struct=8` ablation is withdrawn.** $\mathrm{keep}(s)/\mathrm{keep}(\varphi)$ = 2.2074/2.3464/1.2014, mean 1.9184. The encoder *increases* the within-window variance fraction of what passes through it. §4.2bis's recommendation to drop the encoder was predicated on it being the bottleneck; on this measurement it is not, and running it would cost ∼9 GPU-h to confirm nothing.
- **The measured binding constraint: $\mathrm{keep}(\varphi) = 0.0573$.** Only 5.7% of φ's variance lies *within* a 32,768 bp window; 94% is between windows. φ is defined on 5 kb bins, so a

window spans ∼6.5 bins. The mechanism biases Δ *per position*, and within a window its input is close to constant *by construction of the window size*. This is a sufficient account of the Phase 4 null that does not require the mechanism to be wrong, and it makes the window — not `d_struct` — the next thing to change.

- **Transfer probe (`scripts/phase5_structure_probe.py`, `p5_structure_probe.json`).** Ridge on the mean-pooled final hidden state, 1,200 train / 274 val windows, α by LOO-GCV inside train only. φ is **withheld** from the model at probe time (S0 zeros) and used only as the target, so the structural arm cannot copy its own input. Val Pearson r, mean $\pm$ sd over 3 seeds:

target	GC/dinuc floor	structural	baseline_v2
insulation_100kb	+0.1905	$+0.1208 \pm 0.0283$	$+0.0832 \pm 0.0612$
directionality_2Mb	+0.0918	-0.0093 ± 0.0359	-0.0381 ± 0.0032
compartment_pc1	+0.4511	$+0.5205 \pm 0.0167$	$+0.4896 \pm 0.0179$

The structural arm is ahead on 3/3 targets — the first consistent signal in its favour anywhere in this project. Two caveats that must travel with it. First, on insulation and directionality *both* arms fall below a floor built from GC and dinucleotide content alone, so neither has demonstrated absolute competence on the two targets that most directly test 3D knowledge; the directionality row compares two negative correlations and is a difference between two nulls. Only `compartment_pc1` has both arms above floor, and it is the target most confounded with GC. Second, $n = 3$ per arm: the minimum attainable two-sided permutation p is 0.1000, so nothing here reaches significance and nothing here can.

- **Positional probe (`scripts/phase5_positional_probe.py`, `p5_positional_probe.json`).** 500 train / 274 val windows, positions sampled every 512 bp (64 rows per window; 32,000 train and 17,536 val rows), φ withheld as before. Floor is GC + dinucleotide composition in a $\pm$2,560 bp neighbourhood of each sampled position, put through the identical ridge and identical centring. Two forms: **A** predicts φ_t from h_t; **B** predicts $\varphi_t - \overline{\varphi}_w$ from $h_t - \overline{h}_w$, removing window identity — which keep(φ) = 0.0573 says is 94% of the variance — and leaving exactly the component the Δ-bias mechanism was designed to act on.

target	probe	floor	structural	baseline_v2
insulation_100kb	A	+0.1207	$+0.0644 \pm 0.0065$	$+0.0683 \pm 0.0061$
insulation_100kb	B	+0.0802	$+0.0271 \pm 0.0028$	$+0.0167 \pm 0.0047$
directionality_2Mb	A	−0.0191	-0.0210 ± 0.0065	-0.0212 ± 0.0039
directionality_2Mb	B	+0.0170	-0.0049 ± 0.0015	-0.0027 ± 0.0036
compartment_pc1	A	+0.3396	$+0.1819 \pm 0.0033$	$+0.2283 \pm 0.0237$
compartment_pc1	B	+0.0519	$+0.0089 \pm 0.0047$	$+0.0032 \pm 0.0052$

The one clean arm separation in this project to date is probe B on insulation. Per seed, structural +0.0254/ + 0.0303/ + 0.0255 against baseline +0.0134/ + 0.0146/ + 0.0221: every structural seed exceeds every baseline seed. That is the maximally extreme arrangement at 3 vs 3 and therefore attains the floor of the exact two-sided permutation test, $p = 0.1000$. It is on precisely the quantity the mechanism was built to move, and it is reproducible across seeds.

Two caveats carried in the same breath. First, **both arms sit below the composition floor** on B (+0.0802): local GC predicts within-window insulation better than either trained model does, so the arm contrast is a comparison between two incompetent predictors and demonstrates relative, not absolute, structural sensitivity. Second, the other two targets give nothing — B directionality is negative for every run, and B compartment overlaps between arms (structural min +0.0036 vs baseline max +0.0082).

- **The pooling hypothesis was wrong, and is recorded as wrong.** The positional probe was run because mean-pooling over 32,768 positions was suspected of hiding local insulation structure. It did not: probe A is *lower* than the pooled probe throughout (compartment $+0.1819/+0.2283$ against $+0.5205/+0.4896$ pooled), and on A compartment the baseline arm leads. Pooling was denoising a window-level target, not destroying a local one. The result that survived came from probe B, which was included for a different reason.
- **What a wider window would buy, measured** (`scripts/phase5_window_scan.py`, `p5_window_scan.json`) No model is involved: keep(φ, W) is computed from φ alone, interpolated to token resolution as the model receives it, 300 windows per width each required $\geq$ `MIN_STRUCT_FRAC` valid.

window (bp)	bins spanned	keep(φ)	vs 32,768	signal/compute
8,192	1.6	0.0092	—	—
16,384	3.3	0.0318	—	—
32,768	6.6	0.0490	1.00×	—
65,536	13.1	0.1099	2.24×	1.12
131,072	26.2	0.2117	4.32×	1.08
262,144	52.4	0.3499	7.14×	0.89
524,288	104.9	0.4932	10.06×	0.63

The scan gives 0.0490 at the current width against D4's 0.0573 on the 274 val windows — different samples of the same quantity, consistent. Widening returns slightly more signal than it costs in compute up to 131,072 bp (4.32× signal for 4× step time) and turns sublinear beyond it (7.14× for 8×). **131,072 bp is therefore the indicated target**, with 65,536 as the cheaper derisking run. This is an upper bound on what a wider window can hand the mechanism, not a prediction that the model will use it.

Status of these findings. Three seeds, one chromosome, models not trained to convergence, and the two retractions are a re-analysis of stored numbers rather than a new measurement of the trained system. They are recorded here because they change what should be run next, and because the retracted claims were load-bearing for the Phase 4 verdict. They warrant independent checking before they are carried into a manuscript.

2026-08-16 (third entry) — PHASE 4 COMPLETE: the benefit is null, the pathway is live only in the top four layers, and the permeability gate never moved

All three structural seeds reached `status: COMPLETED` at step 2000. Three measurements were then run back to back on the finished checkpoints, and they do not all say the same thing. What follows is every number, and then what the disagreement between them means.

- **Benefit: null, decisively.** `scripts/phase4_report.py` against `baseline_v2`:

arm	seed	val bits/nt	val acc	τ median
structural	0	1.5266	0.5222	473.8
structural	1	1.5177	0.5256	500.8
structural	2	1.5209	0.5234	487.3
baseline	0	1.5196	0.5247	451.6
baseline	1	1.5172	0.5257	452.2
baseline	2	1.5223	0.5241	400.2

Structural mean 1.5217 (sd 0.0045), baseline mean 1.5197 (sd 0.0025). Difference +0.0020 bits, $+0.80\,\sigma_{\text{real}}$, against a pre-registered floor of $2\sigma = 0.0050$. It **does not clear the floor**, and the sign is the wrong one — the structural arm is nominally worse. Exact two-sided permutation test over all 20 arrangements: $p = 0.6000$. The minimum attainable p at 3 vs 3 is 0.1000, so $p < 0.05$ was arithmetically unreachable by this design regardless of the mechanism; that is stated in the report itself rather than left for a referee.

- **Memory horizon: moved, consistently.** Every structural seed's median τ exceeds every baseline seed's (+52.6 tokens on the means). The mechanism does lengthen memory. It buys nothing on the loss.
- **Reliance (P1 swap, `scripts/phase4_p1_swap.py`): non-zero and very small.** Three seeds, val split, fixed masking seed 1234, 1,344,979 masked positions per pass.

control	Δ bits	KL	argmax flip	KL / KL_{S0}
S0 removed	+0.0000	2.539e-05	0.0052	1.00
S1 shuffled	+0.0001	6.334e-05	0.0073	2.49
S2 shifted 10 Mb	+0.0001	5.528e-05	0.0068	2.18
S3 distance-matched	+0.0001	6.827e-05	0.0075	2.69
S4 sequence-matched	+0.0000	2.874e-05	0.0052	1.13

Kernel floor (same φ, same masking, twice) = 0.000e+00 exactly, so the divergences are real at floating-point resolution and not numerical noise. Masking floor, reported as context only, = 1.321e−01: a φ swap moves predictions roughly **2000× less** than re-masking a seventh of the sequence does.

Two honest caveats on this table. First, Δ is flat everywhere — feeding the model deliberately wrong folding costs it +0.0001 bits, i.e. nothing. Second, the ordering $\text{S3} \approx \text{S1} \approx \text{S2} \gg \text{S4} \approx \text{S0}$ is what one expects if divergence tracks *how far the substituted values sit from the true ones*: S1–S3 preserve φ's marginal and so supply wrong-but-plausibly-scaled values, whereas S0 supplies zeros, which is the standardised mean and therefore the closest substitute available. The ordering confirms that the input reaches the output. It is not by itself evidence of structural understanding.

- **Diagnostics D1–D3 (`scripts/phase4_d1_diagnostic.py`, new this entry) — and this is the finding that matters.** D1 is $\text{Var}_t(W_{\Delta s} s_t)/\text{Var}_t(W_{dt}\delta'_t)$ per layer per direction, with $< 0.05 \Rightarrow$ inert (F1), pre-registered at §4.1.3. Median pooled D1: **0.0004, 0.0002, 0.0002** across seeds 0, 1, 2 — between two and three orders of magnitude *below* the inert threshold. But the median hides the actual structure, which is a sharp depth discontinuity reproduced in all three seeds:

layer	seed 0	seed 1	seed 2
L00–L11 (mean)	≤ 0.0023	≤ 0.0004	≤ 0.0004
L12	0.9056	0.0004	0.0014
L13	0.8310	0.0007	0.3685
L14	0.8703	0.0039	0.4898
L15	0.8227	0.5064	0.3591

Directions clearing the 0.05 threshold: **8/32, 2/32, 6/32** — and in every seed, without exception, *every one of them lies in layers 12–15.* L15 is live in all three. The structural pathway is inert throughout the encoder and live only in a thin slice next to the output head.

- **D3 separates "no gradient reached it" from "what it produces is a constant".** $\|W_{\Delta s}\|_F$ at the endpoint is non-zero in **32/32 directions in every seed** (per-seed min

0.1048 / 0.0936 / 0.1307, median 0.1943 / 0.1956 / 0.2169, max 0.6614 / 1.0843 / 0.6125), against the exact 0 it was initialised to. So gradient pressure reached the dead layers and moved their weights; what those weights then produce is a term whose variance across positions is $\sim 10^{-4}$ of Δ's. That is failure mode **F1 (bias absorption into b_{dt}) confirmed by direct measurement** in layers 0–11, not inferred. Only the endpoint is available — `train.py` overwrites `checkpoint.pt`, so the trajectory §4.1.3 asks for is not recoverable and this entry does not claim it.

- **D2: the permeability gate never engaged (F5).** p_t initialises at softplus$(-4) = 0.018150$. Measured over 287,309,824 gate evaluations per seed, the mean is **0.017234 / 0.017193 / 0.017108** and the full observed range is $[0.0133, 0.0197]$. All mass falls in the single histogram bin $[0, 0.0312)$. The weights did move — the fraction sitting exactly at the init value is 5.07e-04, 8e-06 and 0 respectively — but never further than about 5% of the init value. The "soft reset at a domain boundary" half of the mechanism did not happen.
- **The two pre-registered criteria now disagree, which §4.1.3 anticipated in writing.** The spec says: "D1 < 0.05 and $D_{S1} \approx 0$ should agree, and disagreement between them is itself informative." They disagree. D_{S1} is non-zero against an exactly-zero kernel floor, which by the letter of the amended gate is the flat-loss / live-divergence cell and licenses proceeding to Phase 5. D1 is INERT at the median in all three seeds. Both are correct, and together they reconcile into one statement rather than a conflict: the 2–8 live top-layer directions are what generate the small non-zero divergence, and the 24–30 dead encoder directions generate none. That is exactly why the divergence is real but sits at 0.05% of the masking floor.
- **What this does to the project's central claim, stated plainly.** The claim is that conditioning *representation learning* on structure beats bolting structure on afterwards — the contrast with CHROME, which trains its sequence encoder to convergence with no structure present. A mechanism that is inert through layers 0–11 and live only in layers 12–15 is, functionally, a late-stage readout correction. It sits much closer to the post-hoc integration this project defines itself against than the design intended. No result is withdrawn and no number here is adjusted; but the mechanism as built and trained did not do the thing the claim rests on, and any write-up must say so.
- **Not concluded, deliberately.** This is one configuration: $d_{\text{struct}} = 2$ behind an encoder that §4.2bis measured training roughly five orders of magnitude slower than $W_{\Delta s}$, at 2000 steps with val loss still descending in both arms. F1 confirmed in the encoder is consistent with an under-parameterised, slowly-trained s_t that early layers cannot use as anything but a bias. Whether the depth profile survives dropping the encoder for $d_{\text{struct}} = 8$ (the spec's own §4.2bis recommendation, +1.70% params, not applied to these runs) is untested and is the cheapest informative next experiment. Recording it as untested, not as refuted.

Artifacts: `results/novel_model/phase4_report.txt`, `results/novel_model/p1_swap_results.json`, `results/novel_model/d1_diagnostic.json`.

2026-08-16 — Controls S3 and S4 built; the P1 gate is pre-registered, and its first draft was wrong

- **Two decisions of record added (§7, items 5 and 6), both PI decisions, both recorded before any control was fed to any trained model.** At the time of writing, Phase 4 seed 0 was `COMPLETED`, seed 1 was mid-run, seed 2 had not started, and no swap artifact of any kind existed. That is checkable against the run directories and the git history, and it is what makes this pre-registration rather than an adjustment.
- **Decision 5 — reliance is measured separately from benefit.** The P1 gate as originally written read reliance off a *loss delta*. Masked-token prediction over a 32 kb

window is dominated by local k-mer statistics, so a flat Δ_{S1} is consistent with three incompatible states: the mechanism is inert; the mechanism is used but MLM cannot express its contribution; structure carries no information. §4.1.3 assigned all three the same verdict — "do not proceed to Phase 5" — and Phase 5 is where the hypothesis lives. Evidence the metric lacks power: correcting `dt_min` lengthened the trained memory horizon 30× and moved validation loss 0.0013 bits. Reliance is now prediction divergence, $D_S = \text{mean}\,\mathrm{KL}(p_{\text{real}} \,\|\, p_S)$ over masked positions, plus the argmax-flip rate F_S. Benefit is unchanged: $\Delta_{S1} \geq 2\sigma_{\text{real}} = 0.0050$ bits with a 95 % CI excluding 0.

- **The first draft of that rule was wrong, and is recorded as withdrawn rather than quietly replaced.** It set the pass threshold at "exceed the divergence from re-evaluating under a different masking seed". A dry run of `scripts/phase4_p1_swap.py` against a deliberately untrained 20-step model — executed before any trained checkpoint was swapped — showed two faults. *The magnitudes are mismatched*: re-masking perturbs 15 % of the nucleotides the model reads, while a φ swap perturbs an auxiliary channel, so the rule demanded that lying about the folding move predictions further than re-masking a seventh of the sequence does. Measured, that floor was KL = 19.56. *And it was unnecessary*: a model that ignores φ does not produce a small divergence, it produces bitwise identical logits. On the dry-run model, whose $W_{\Delta s}$ was still at its zero init, every control returned KL $\approx 10^{-23}$ with an argmax-flip rate of exactly 0.000000 against a kernel floor of exactly 0.0. Corrected rule: **reliance passes iff D_{S1} exceeds the kernel floor by a wide margin ($> 10^{-12}$) with a materially non-zero flip rate**. The masking floor is still computed and reported, labelled context-only. D_{S0} is reported as the upper reference, so D_{S1}/D_{S0} states how much of the available structural effect the shuffle disrupts.
- **Decision 6 — control S4 `SEQUENCE-MATCHED` added.** `compartment_pc1` correlates strongly with GC content and gene density, both computable from sequence alone, so without an *aligned* 1D control a positive result reads as "you handed the model a GC proxy". S2 cannot cover this: the objection requires alignment to be preserved and S2 destroys it. S3 and S4 together remove the two benign explanations — genomic distance, and sequence composition.
- **S4 built** (`scripts/phase4_build_s4.py`, CPU only, no Hi-C read at any point). Eight aligned covariates from `tokens_chr9.npy` and GENCODE (3,190 genes on chr9): GC smoothed at 100/250/500 kb, a 2 Mb GC gradient, gene density, upstream-minus-downstream gene fraction, CpG-over-GC, and a GC+gene composite. Reproduces φ's `feature_symmetry` $[1, 1, 1, -1, 1, -1, 1, 1]$ — asserted, with the two antisymmetric coordinates built as downstream-minus-upstream differences so they genuinely flip under reverse-complement — and standardised chromosome-wide exactly as φ is, over the same **21,519** usable bins.
- **Measured, and it settles the objection quantitatively.** Best correlation of each φ coordinate with any S4 coordinate: `compartment_pc1` r = **0.7384** with `gc_500kb`, i.e. smoothed GC explains **55 %** of its variance. Every other coordinate is at $|r| \leq 0.19$ (worst: `insulation_100kb` at 0.1822). **The GC objection was real but confined to one of eight features**; insulation and directionality — the coordinates that encode domain boundaries — are not recoverable from sequence composition.
- **S3 built** (`scripts/phase4_build_s3.py`, CPU only; both Hi-C inputs were already cached under `data/interim/`, so no network). Contact values are permuted *within each diagonal* of the 5 kb band, and on the upper triangle of the 250 kb matrix with symmetry restored by mirroring. Permuting within a diagonal — rather than substituting the diagonal mean — preserves the value distribution as well as the mean, so the control is not testing two things at once. φ is then recomputed by calling `phase1_features`' *own* functions, so the only difference from real φ is the matrix it was computed from.
- **S3 verification.** $P(s)$ preserved to 1.86×10^{-9} (max absolute deviation of any diagonal

mean, over 401 diagonals); 8,612,548 measurable cells permuted with 2,486,731 NaNs left in place, so the measurability pattern is untouched; **21,519** usable bins, identical to real φ. Correlation of each recomputed feature with its real counterpart: worst $|r| = 0.1818$, `compartment_pc1` at -0.0448. Distance decay preserved exactly, locus specificity gone.

- **Test suite extended to 39/39** (`test_phase4_wiring.py`). The old assertion that S3 raises when its file is absent was removed — it stopped being true once S3 was built — and replaced with checks that both file-based controls match φ's shape, are defined at *exactly* the same bins, differ from real φ, and satisfy their defining properties. Coverage equality matters: a control defined at a different set of positions would confound the comparison with coverage. `test_model.py` still 17/17.
- `CLAUDE.md` **rewritten** as a current-state working document; the original pre-research plan is preserved unchanged at `docs/original_plan.md`. A plain-language summary for non-specialist readers was added at `docs/project_summary_plain.md`. `train.py` gained `--out-dir`; `phi_control` gained shape and coverage validation for file-based controls. `build_project_doc.py` now decodes pdflatex output with `errors="replace"`.
- **Validation reports are now tracked in git** (`.gitignore` exception for `data/processed/*_validation_rep`). They carry measured numbers the write-up will cite — φ's agreement with the independent 4DN tracks, S3's $P(s)$ preservation, S4's correlation with φ — and a number a paper quotes has to live in a tracked file.

2026-08-16 (second entry) — val/test windows were a thinned train grid; now enumerated independently at the no-overlap stride

- **The bug.** `build_index` scanned the chromosome once, at the training stride `STRIDE_TRAIN = 16,384`, and kept a held-out window only when `start % STRIDE_EVAL == 0`. So `STRIDE_EVAL = 32,768` was a *filter* on a grid anchored to the chromosome origin, not an independent enumeration: every val/test window had to land simultaneously on both the half-window train grid and the full-window eval grid, and the region was covered from whichever global multiple happened to fall inside it. The held-out windows did not overlap (verified: all consecutive val and test gaps exactly 32,768), but the enumeration was an emergent property of the train grid, not a stated one.
- **The fix.** val and test are now enumerated in their own pass, anchored at each region boundary and stepping by `STRIDE_EVAL == WINDOW`: `range(region[0], region[1], STRIDE_EVAL)`. Held-out windows still must sit *entirely* inside their region (`assign_split`), so the buffer rule is unchanged, and the same N-fraction and structure-fraction filters apply. The train scan is untouched.
- **Re-reported counts** (rebuilt `dataset_index.npz`, 2026-08-16, no model affected — `train.py` holds the old index in memory for the running seed):
  - train: **5,422** (unchanged, identical starts),
  - val: 273 → **274**, test: 242 → **243**.
  - no overlap in either held-out set; every consecutive gap exactly 32,768.

2026-08-15 (second entry) — PHASE 4 STARTED: the structural arm is wired and pretraining

- **What was missing.** The structural *model* has existed and passed its unit tests since Phase 2 (7,758,354 parameters, +0.43 %). What did not exist was the *data path*: `train.py` hardcoded `structural=False`, its `collate` dropped φ, and its checkpointed forward raised `NotImplementedError`. That is what this change builds.
- **Wiring.** `collate` now carries `phi` and `phi_valid` (with `np.ascontiguousarray`, because the reverse-complement augmentation returns negative-stride views that `torch.from_numpy`

rejects); a single `batch_struct` helper supplies the structural kwargs to the training loop, the eval loop and the τ probe so the three cannot drift apart; `CheckpointedLM.forward` mirrors `BiMambaLM.forward`; and `--structural`, `--phi-control` and `--out-dir` are exposed on the CLI.

- **τ measurement corrected for the structural arm.** The structural path adds `W_dstruct(s)` to `dt_pre` *before* the softplus, so `DeltaCapture`'s hook on `dt_proj` alone would have reported the **baseline's** τ for the structural model. `DeltaCapture` now hooks `W_dstruct` as well and sums the two. This would have been a silent error, not a loud one: the numbers would have looked entirely plausible.
- **Shuffled-structure controls implemented** (§4.1.3), in `phase1_dataset.py::apply_phi_control`. S1 `GLOBAL-PERM` and S2 `CIRCULAR-SHIFT` (10 Mb) are pure φ transforms. S3 `DISTANCE-MATCHED REWIRE` cannot be: it requires resampling contacts under the empirical $P(s)$ and recomputing φ, so it raises a `FileNotFoundError` naming the script that must build it. Controls draw from a **fixed** seed (20260815), deliberately not the training seed — every training seed must see the *same* shuffled structure or seed variance and shuffle variance are confounded and the $2\sigma_{\text{real}}$ gate cannot be read. A control requested without `--structural` is a hard error, since the baseline never reads φ and the run would be mislabelled.
- **Verification before compute:** `scripts/test_phase4_wiring.py`, **27/27 passing**. It checks that S1/S2 preserve the marginal exactly, that S2 preserves local autocorrelation while S1 destroys it (successive-difference sd 0.2608 $\rightarrow$ 1.2471), that φ arrives as $(b, 32768, 8)$ float32 with invalid positions exactly zero, that gradient reaches both the encoder and `W_dstruct`, that `DeltaCapture` sees the structural contribution, and — CLAUDE.md's Phase 4 gate — that **shuffling φ changes the output** once `W_dstruct` is non-zero, while at initialisation it provably does *not* (zero-init equivalence). `test_model.py` still passes 17/17.
- **One test tolerance was wrong, not the code.** The S2 autocorrelation check was written at `rtol=1e-6` and failed at 1.2×10^{-4}. The residual is the single bin-to-bin transition `np.roll` replaces with the seam across 27,679 bins — the control working as designed. Tolerance corrected to `1e-3`, with the reason recorded at the assertion.
- **A bug caught in the first launch, four minutes in.** `train.py` hardcoded `RESULTS = results/baselines`, so the Phase 4 run wrote there while `run_phase4.sh` watched `results/novel_model` for `status: COMPLETED`. The run would have trained correctly and then been retried until the 20-attempt limit, because the supervisor could never see it finish. Fixed with `--out-dir`; the launch was stopped, the stray run directory removed, and an orphaned spawn child holding port 29511 killed before relaunching.
- **Now running.** `results/novel_model/structural_seed{0,1,2}`, started 2026-08-15 21:38 IST, supervised by `run_phase4.sh` + `phase4_guard.sh` (same design as Phase 3, same `--ckpt-every 120` against the idle culler). **Every training hyperparameter is identical to Phase 3's**, because matched compute is the basis of the comparison; the only differences are `--structural` and the output directory.
- **What is deliberately NOT running.** §4.1.3 splits Phase 4 into P1 (swap φ at inference on the real-structure model; cheap; *this is the gate*) and P2 (pretrain separate S1 and S3 models; six seeds, $\sim$18 GPU-hours). Only the three real-structure seeds ($\sim$9 GPU-hours) are launched. If P1 shows the mechanism is inert, the spec says stop — and P2 would have been 18 GPU-hours spent answering a question that no longer matters.
- **The bar.** P1 passes iff $\Delta_{S1} > 0$ with a 95 % bootstrap CI over seeds excluding 0 **and** $\Delta_{S1} \geq 2\sigma_{\text{real}} = \mathbf{0.0050}$ bits. For scale, the three Phase 3 v2 seeds span 1.5172–1.5223.

2026-08-15 — PHASE 3 RE-RUN COMPLETE: the F4 gate passes on trained τ

- **What ran.** Three seeds at the corrected Δ initialisation, `results/baselines/baseline_v2_seed{0,1,2}`, 2000 steps each, same recipe, data and parameter count as the original `baseline_seed*` runs. Seed 2 finished 2026-08-15 20:42:38 IST. Report: `results/baselines/phase3_report_baseline_v2.txt` generated by `scripts/phase3_report.py` from `metrics.json`; no number below was typed by hand.
- **Result, three seeds at step 2000.** Validation 1.5196 / 1.5172 / 1.5223 bits per nucleotide, **mean 1.5197, sd 0.0025**; accuracy mean 0.5248, sd 0.0008; uniform-random floor 2.0000, so the improvement on floor is 0.4803 bits.
- **The F4 gate, read from empirical τ on real validation batches.** Median 434.7 $\pm$ 29.8 tokens; p90 47,680 $\pm$ 480; p99 358,615 $\pm$ 22,332. Exact fraction of (position, channel, state) triples with $\tau \geq$ 5,000: 0.2928. Exact fraction with $\tau \geq$ 100,000: $4.846 \times 10^{-2} \pm 1.056 \times 10^{-3}$, against the PI-fixed floor of 10^{-4} — a margin of 485×. All **32 of 32** layer-directions contain triples at TAD scale, against 0 of 32 before. **Verdict: proceed to Phase 4.**
- **Against the old architecture, at identical parameter count (7,725,312).** Validation 1.5210 $\pm$ 0.0040 $\rightarrow$ 1.5197 $\pm$ 0.0025; median τ 14.2 $\rightarrow$ 434.7; TAD-scale mass $\sim$0 $\rightarrow$ 4.85×10^{-2}.
- **Finding worth carrying forward: the 30× longer memory horizon bought nothing on masked-token prediction.** 1.5210 $\rightarrow$ 1.5197 is well inside seed noise. This is not a failure — MLM over 32 kb windows is dominated by local sequence statistics — but it removes the convenient story that longer memory improves pretraining, and it means the structural arm must earn its result on downstream tasks rather than on validation loss.
- σ_{real} **is now 0.0025, not 0.0040.** The Phase 4 gate in `architecture_spec.md` §4.1.3 is $\Delta_{S1} \geq 2\sigma_{\text{real}}$, so the bar is **0.0050 bits** — tighter than before, and comparable to the spread between seeds 1 and 2 (1.5172 vs 1.5223).
- **Scope limit now written into the spec.** The training window is 32,768 bp, so $\tau \geq$ 100,000 is 3× the window and a 385 kb TAD is 11.7× it: within one forward pass no τ beyond $\sim$32,768 is behaviourally distinguishable from any larger one. The supported claim is *the state retains across the full window* — at median $\tau = 14$ a feature spanning one 5 kb bin was inexpressible; at 435 with 4.8 % of states effectively non-decaying it is not. The model was never required to *see* a TAD; φ carries the long-range structure. `test_model.py`'s `tau_max >= 385_000` assertion is retained as a regression guard on the Δ initialisation, not as evidence of TAD-scale modelling capacity.
- **Documentation corrected to match.** `architecture_spec.md` (superseded-notice on the pre-fix F4 table; the "what the tests guarantee" paragraph still claimed $\tau_{\max} \approx$ 1000, which the current assertions contradict; new "Re-run complete" subsection; new scope-limit subsection), `data_card.md` and `gpu_runbook.md` (dated resolution notices), and `docs/primer.html` / `primer_print.html` (the "one open result" callout rewritten as a resolved one). **The primer PDF is stale**: no Chrome or Edge is installed on this machine, so `scripts/build_primer_pdf.py` cannot render it here.
- **Provenance bug fixed** in `train.py::git_state`. `subprocess` output was `.strip()`ed before parsing `git status --porcelain`, which ate the leading blank status column of the *first* line only, so `ln[3:]` cut one character too many and runs recorded `esults/...` instead of `results/...`. Every later line was correct, which is why it survived until the v2 configs were diffed. The porcelain call now preserves indentation.

2026-08-12 — F4 resolved: the cap was `dt_min`, and it was exact

- **Cause, in one line of code.** `model.py::_init_dt` draws Δ log-uniform over $[\texttt{dt\_min}, \texttt{dt\_max}] = [10^{-3}, 10^{-1}]$, and `model.py` line 128 sets $A = -\text{arange}(1, \texttt{d\_state} + 1)$ so $|A| \geq 1$. Since $\tau = 1/(\Delta|A|)$, $\tau_{\max}$ **at initialisation is exactly** $1/\texttt{dt\_min} = 1{,}000$ **tokens**. Phase 3 measured 999.5. Mamba's reference `dt_min` was a hard ceiling on the memory horizon 100× below TAD scale, and training never moved the bulk: median τ 14.6 at init, 14.2 after 2000 steps. The architecture could not express what the mechanism conditions on.
- **Change:** `dt_min` $10^{-3} \to 10^{-6}$, `dt_floor` $10^{-4} \to 10^{-7}$, `dt_max` unchanged. Promoted from `_init_dt` defaults to `ModelConfig` fields so `train.py` records them in `run_config.yaml`: a run whose memory horizon is missing from its own provenance cannot be compared with another.
- **Measured at init, not asserted** (`f4_memory_horizon.py`, corroborated by `tau_stats()` on the instantiated model): median τ 14.9 → 472.5; p90 108.8 → 47,940; p99 497.4 → 290,487; $\tau_{\max}$ 994 → 985,872. Fraction at TAD scale 0.0000 → 0.0433, which is 433× the 10^{-4} gate floor. The 385 kb validated TAD now fits inside $\tau_{\max}$ with 2.5× headroom.
- **Parameters unchanged at 7,725,312.** This is an initialisation change, not a capacity change, so the matched-compute comparison with the structural arm survives intact, as does structural/baseline init equivalence (`test_model.py`, max gap still exactly 0.0). 17/17 checks pass.
- **Why this lever and not the obvious ones.** Raising `d_state` adds states but $|A|$ still starts at 1, so the ceiling does not move. Raising `d_model` draws more channels from the same Δ range and breaks the parameter match. Widening the Δ range is the only lever that moves the ceiling at fixed capacity. Re-initialising A log-spaced over $[0.01, 16]$ is an independent lever that also works (measured $\tau_{\max}$ 99,432 on its own) and was deliberately *not* applied — one variable at a time, so a Phase 4 result can be attributed to something.
- **The `dt_floor` clamp is a trap, and it was nearly walked into.** `_init_dt` clamps Δ at `dt_floor`, capping τ at $1/\texttt{dt\_floor}$ whatever `dt_min` says. Lowering `dt_min` to 10^{-6} while leaving `dt_floor` at 10^{-4} gives $\tau_{\max}$ of exactly 10,000 tokens and reads as "the change did nothing". `_init_dt` now raises `ValueError` instead of capping silently, and `test_model.py` asserts $\tau_{\max} \geq 385{,}000$.
- **This invalidates the Phase 3 baseline for comparison purposes.** The three completed seeds were trained at `dt_min`$= 10^{-3}$ and are a different architecture. They stand as the record of *why* this change was made, but they are not the Phase 4 baseline and their $\sigma_{\text{real}} = 0.0040$ does not carry over. Phase 3 must be re-run on the new initialisation, and the F4 gate re-read from *trained* τ. The init numbers establish that TAD scale is now expressible, not that training preserves it — and Phase 3's central finding was exactly that init τ and trained τ diverge.

2026-08-12 — PHASE 3 COMPLETE: the F4 gate fails, Phase 4 must be re-scoped

Three seeds × 2000 steps of `BiMambaLM(structural=False)` on the chr9 pilot, all `COMPLETED`. Every number below is read by `scripts/phase3_report.py` out of `results/baselines/*/metrics.json` and `run_config.yaml`; the report is committed at `results/baselines/phase3_report.txt`.

- **The baseline trains, and the three seeds agree.** Final validation 1.5210 bits/nucleotide, sd 0.0040 across seeds (1.5164 / 1.5239 / 1.5227), accuracy 0.5240 ± 0.0014 — 0.4790 bits below the 2.000-bit uniform floor. **That sd 0.0040 is σ_{real}**, the quantity §4.1.3 needs for Phase 4's 2σ effect-size gate; it is now measured rather than assumed.
- **The F4 gate FAILS. Trained τ does not approach TAD scale.** Median τ 14.2 ± 0.2 tokens, p99 574.8 ± 55.3 — essentially unmoved from the 14.6 median measured at

initialisation on 2026-08-07. The distribution's bulk never left the degenerate regime.

- **The tail is wildly seed-dependent and that is itself the finding.** $\tau_{\max}$ is 46,319 ± 61,333 tokens: a standard deviation larger than the mean. Per seed it is 10,584 / 117,139 / 11,235 — seed 1 alone crossed 100 kb, and only in `layer15.rev`, one of 32 layer-directions. Averaged over seeds, 0.3 of 32 layer-directions contain any triple at TAD scale. A single-seed run would have given either "comfortably short" or "crosses TAD scale" with equal ease. This is exactly the case the three-seed requirement exists for.
- **Gate arithmetic, against the criterion fixed earlier today:** mean $\tau_{\max}$ 46,319 (needs 100,000) — fail; mean relay heuristic 77,655 ± 51,512 (needs 100,000) — fail; exact mass at TAD scale $6.954 \times 10^{-8} \pm 1.204 \times 10^{-7}$ (needs 10^{-4}) — fail by ∼1400×. All three arms fail independently, so the verdict does not rest on the threshold decision alone.
- **Action, per the §4.1.4 decision table: re-scope to sub-TAD structural signal, or raise `d_model`/`d_state`, before spending Phase 4 compute. This is a statement about the backbone, not about the structural mechanism**, and it constrains the baseline and the structural arm identically: a model that cannot carry information 100 kb cannot be helped by being told where the 100 kb boundaries are. It does not say structure is useless—φ carries sub-TAD signal too, and the interpolated s_t varies at every position—it says the TAD-scale framing of the claim is not supportable at this model size.
- **Why `tau_stats()` alone would have misled.** Bias-only $\tau_{\max}$ after training is 1,050.7 ± 21.0 tokens, against an empirical 46,319. The runbook asked for `tau_stats()`; had the gate been read from it, the answer would have been off by a factor of 44, because once trained, Δ is dominated by the input-dependent term $W_\Delta \delta'_t$ rather than by the bias.
- **Cost.** Six idle-culler kills across two days. No computation was ever lost—every resume came off a checkpoint—but the last of them stalled progress entirely once the cull interval fell below the 200-step checkpoint interval, and `--ckpt-every` was cut to 120 (PI's choice) to fit inside it. Seed wall-clock figures in the report are per final attempt and understate the interrupted seeds.

2026-08-12 — The F4 gate was unmeasurable as written, and is now pinned

- **The gate's mass test could not see the effect it tested for.** `empirical_tau` computes the summary's `tau_max` as an exact maximum over the full τ tensors (`train.py:362`) but its `frac_ge_*` from a random subsample of 20,000 per layer-direction (`train.py:352`). Different populations, so the two can disagree, and did: seed 1 at step 1000 logged `tau_max` 103,025 beside `frac_ge_100k` of exactly `0.0`. That is a detection floor, not a contradiction—the subsample cannot resolve anything rarer than $1/20{,}000 = 5 \times 10^{-5}$, and the true value was 5.96×10^{-8}, some 840× finer. Read from that field, the gate would have answered "no mass at TAD scale" by construction rather than by measurement, for any model in this regime. `phase3_report.py` now reads the exact per-layer fractions, which are computed on the full tensors and were already logged; no re-run was needed. `train.py` was deliberately *not* edited—it was mid-sweep, and changing it would have measured seed 2 with different code than seeds 0 and 1.
- **The gate's threshold was qualitative and is now operational (PI decision).** §4.1.4 said "τ reaches $\gtrsim$100 kb"; the data made the ambiguity load-bearing. The gate now passes iff mean $\tau_{\max} \geq 100{,}000$ tokens, or mean relay heuristic $\geq 100{,}000$ *and* exact `frac_ge_100k` $\geq 10^{-4}$. The mass floor is the new part. The prior condition was "any nonzero mass", which seed 1 met at 1.19×10^{-7}—about four triples per million, in 1 of 32 layer-directions, while median τ was 14.4 tokens and the other 31 layer-directions held nothing at TAD scale.
- **This mattered numerically, not just in principle.** On the data logged at the time of the decision, the mean relay heuristic stood at 98,636 tokens, 1.4% under its threshold,

and it is the fastest-moving term in the gate (23,844 → 151,021 across seed 1). Under the old rule seed 2 could have carried it over the line and flipped the verdict to "proceed to Phase 4" on the strength of a quantity §4.1.4 itself calls "a HEURISTIC and not a bound". Under the new rule the mass floor binds and misses by roughly 1,700×.

- **Fixed before the evidence was in.** The threshold was set while seed 1 was at step 1200 of 2000 and seed 2 had not started, and before any verdict was computed. Recorded explicitly because a gate loosened or tightened after seeing its own answer is not a gate; the ordering is the whole point.

2026-08-11 — Phase 3 run supervision

The Phase 3 baseline is three seeds × 2000 steps at a measured 5.30 s/step, i.e. roughly 2.9 h per seed and 8.8 h in total. That exceeds the lifetime of an interactive session on this cluster, so the runs were made independent of one. Training results are recorded in a later entry.

- **The first launch was killed by session teardown** at step 200 of seed 0. Background processes started from an agent session die with it. Runs are now started with `setsid nohup`, giving them their own session, no controlling terminal and `init` as parent. Confirmed independently that this box has `KillUserProcesses=no` and no batch scheduler, so nothing reaps user processes at logout. **This was necessary but not sufficient, and the conclusion drawn from it at the time was wrong**—see the second entry for 2026-08-11, where the run was killed anyway by a mechanism none of these checks covered.
- `scripts/run_phase3.sh` **added** as the supervisor. It lives in the repository rather than a temporary directory—bash reads a script incrementally as it executes, so a driver under a path that is cleaned up mid-run can fail partway through the seed loop. It retries a failed seed up to 20 times with `--resume`, and decides completion from `status: COMPLETED` in the run's own `run_config.yaml`, which rank 0 writes only after the training loop exits normally. Deciding from an exit code instead would let a half-finished run be recorded as finished. A `flock` on fd 9 makes a second instance impossible: two supervisors would have two training processes writing the same `checkpoint.pt`.
- `scripts/phase3_guard.sh` **added** to restart the supervisor if it is killed outright. There is no writable user crontab and no `systemd --user` on this box, so it is a plain detached poller and does *not* survive a machine reboot; the entry point after a reboot is a single command, documented in the script's header.
- **Liveness must not be probed with `pgrep -f`.** The guard's first version used `pgrep -u $USER -f run_phase3.sh`, which matches any command line that merely mentions the script—an editor, a grep, the very command that launched the guard. It returned a false positive on its first poll and left training stopped. Liveness is now read from a pidfile and confirmed against that pid's `/proc/<pid>/cmdline`.
- `build_project_doc.py` **was silently failing its own rule check.** The script already returns exit 1 when `project-docs/` holds anything besides `project.tex` and `project.pdf`, but JupyterLab recreates a `.ipynb_checkpoints/` directory there within minutes of any build, because it holds the `.tex` open. The exit code had been read through a pipe to `tail`, which reports the exit status of `tail` rather than of the script, so the violation went unnoticed across two builds. The cleanup loop now removes that directory, and the exit code is read directly. It contained only Jupyter's autosave copies of files the script rebuilds, so nothing unique was discarded—verified before the first deletion by diffing the checkpoint against the live `.tex`: zero lines present only in the checkpoint.
- **The console log lags.** `train.py`'s output is piped through `grep -v "Can't initialize NVML"` inside the supervisor, and grep block-buffers when its stdout is a file, so training

lines can appear tens of lines late. The supervisor's own markers bypass the pipe and are prompt. `scripts/phase3_progress.py` was added to read progress from `metrics.json`, which `train.py` writes atomically at every eval and which never lags. Adding `--line-buffered` to the supervisor's grep is deferred until the runs finish, because editing a script that bash is currently executing can corrupt its parse.

- **Checkpoint durability confirmed by reading the code:** `save_ckpt` writes to `.tmp` and then `Path.replace`, which is `os.replace` and atomic within a filesystem, so a kill during a write cannot leave a corrupt checkpoint. `metrics.json` is written the same way. At `--ckpt-every 200` the worst case is under 18 minutes of lost work.
- **Recovery verified under a real `SIGKILL`, not by inspection.** The trainer was killed outright at step 247 of seed 0 (accidentally, by a `pgrep -f` that matched the killing shell's own command line—the same false-match class as the guard bug above, and the reason the guard no longer uses `pgrep`). The chain behaved as designed: the step-200 checkpoint survived, the supervisor recorded `ProcessExitedException ... signal SIGKILL` and `attempt 1 exit=1`, declined to mark the run complete, and started attempt 2 sixty seconds later. Resume was then confirmed two ways: the step-200 record was still present in `metrics.json` with step 400 *appended* to it, where a restart from scratch would have reset the file to a single fresh row; and attempt 2 reached the step-400 eval 18.2 minutes after starting, which is 200 steps at 5.30 s/step, not the 400 steps a from-scratch run would have needed. Worst-case loss from an interruption is therefore one checkpoint interval, under 18 minutes, with no manual intervention.
- **Resume is not identical to an uninterrupted run.** Model, optimizer, scheduler, step count and RNG state are restored; the dataloader's position within the epoch is not. An interrupted seed therefore sees a different window order than an uninterrupted one. Recorded here so that a seed-to-seed discrepancy is not later mistaken for a real effect.
- **The 8.8 h was not shortened, deliberately.** Cutting 2000 steps to 1000 would halve the wall clock at no engineering cost, but the phase exists to measure *trained* τ for the F4 gate, and τ grows during training. Undertraining biases τ downward and would manufacture a false "TAD scale is not expressible, re-scope Phase 4" verdict. Confirmed with the PI that there is no deadline requiring the trade.
- **No further speedup is available cheaply.** The measured levers are exhausted: `num_warps=1` (1.42×, previous entry), two GPUs at 2.00× scaling, and a batch size already at saturation (batch 4 OOMs, and batch 8 with recompute is 13% worse per window than batch 2 without). Running the three seeds concurrently on one GPU each gains nothing, since DDP already scales at exactly 2× and the total work is fixed. The remaining real lever is a chunked parallel-prefix rewrite of the scan, which is currently a strictly sequential 32,768-iteration chain; it is deferred until after the F4 gate, since the gate may re-scope Phase 4 altogether.

2026-08-11 (second entry) — The idle culler, and why unattended runs are impossible on this box

The supervision built in the previous entry was tested against process death and survived it. It was never tested against the thing that actually kills runs here, and the run was lost a second time as a result.

- **What happened.** Seed 0 was killed at step 1000 while unattended. The console log ends in bash's `Terminated`, i.e. `SIGTERM`; the machine had not rebooted (uptime 126 days); the last checkpoint was written at 04:53:15 UTC, about ten minutes after the browser was closed. Both GPUs were free at 07:16 and the guard, supervisor and trainer were all gone—the entire chain, not one link of it.

- **Cause, confirmed by direct observation.** The hub runs `jupyterhub_idle_culler --timeout=600 --cull-every=60 --concurrency=5 --max-age=0`, and the whole user environment is a single systemd cgroup: `/proc/self/cgroup` reads `0::/system.slice/jupyter-238w1a5447.serv` Ten minutes after the last browser activity the hub stops that service, and systemd kills every process in the cgroup. `setsid`, `nohup` **and** `PPID=1` **give no protection against this.** They defend against `SIGHUP` and against a dying parent; a cgroup-wide kill is neither. The previous entry verified session independence and concluded the run was safe, which did not follow—cgroup membership was never checked.
- **Cost of the incident: wall clock only.** Restarted 07:17:00 from the step-1000 checkpoint. The step-1200 eval appended to the existing records at `val 1.5388` bits, continuing down from step 1000's `1.5671` rather than restarting near the untrained `4.1004`. Resume works; roughly 2.4 h of idle wall clock was lost and no computed result was.
- **Consequence for planning: a 9-hour job cannot run unattended here.** No amount of process supervision fixes a 10-minute idle timeout, because the supervisor is inside the cgroup being killed. The options are to ask the administrators to raise the timeout or provide a batch queue, to keep a browser tab connected, or to accept periodic culling. A heartbeat that pokes the Jupyter API would defeat a resource policy the administrators set deliberately and is not being used.
- `scripts/phase3_status.sh` **added**—one read-only command reporting whether the chain is alive, every eval logged so far, per-seed completion, and GPU memory through the CUDA API (`nvidia-smi` is broken on this box). When nothing is running it prints the command to restart.
- `scripts/phase3_autostart.sh` **added**—idempotent restart. It exits immediately if a guard is already running or if all seeds are done, and otherwise starts the chain, which resumes the current seed from its last checkpoint. Liveness is read from a pidfile confirmed against `/proc/<pid>/cmdline`, not `pgrep -f`, for the reason in the previous entry.
- `scripts/jupyter_server_config.py` **added as an inert template, *not* installed.** Copied to `~/.jupyter/jupyter_server_config.py` it would run the autostart script at every single-user server start, so that reconnecting is itself the restart: `jupyterhub-singleuser` is launched with no `--config`, and `~/.jupyter` is first on the config search path (`jupyter_server` 2.14.1). It does not touch the culler and does not keep the server alive; it only removes the manual restart on return. It is left uninstalled pending the PI's decision, and is untested—the hook can only be exercised by a real server restart, and a parse error in that file would prevent the server from starting at all.
- **Times are displayed in IST** (`Asia/Kolkata`, +05:30, no DST) in `phase3_status.sh`, `phase3_autostart.sh` and `phase3_report.py`, at the PI's request. **Stored timestamps remain UTC**—`train.py` writes `completed_at_utc`, and a recorded time should not change meaning with the reader; only the display converts. Verified on the real record rather than a synthetic one: `2026-08-11T09:13:03+00:00` renders as `2026-08-11 14:43:03 IST`. `run_phase3.sh` and `phase3_guard.sh` still log UTC and were left alone deliberately: both were executing, and bash reads a script incrementally as it runs, so editing one can corrupt its parse mid-run.
- **Wall clock is measured per attempt, not per seed.** `train.py` times from the start of the attempt that reached the final step, so an interrupted seed under-reports its true cost. Seed 0 was interrupted twice and records 6957 s, which is its final attempt only. `phase3_report.py` now prints this caveat beside the figure. No result depends on wall clock.

2026-08-10

First session on the L40S box. Environment built, pipeline reproduced from a clean tree, custom kernel executed for the first time. No model trained.

- **Environment.** `torch 2.6.0+cu124`, `triton 3.2.0`, Python 3.10, venv `3d-gen/` (added to `.gitignore`). `torch.cuda.device_count() = 2`; both devices report NVIDIA L40S, compute capability 8.9, 44.4 GiB.
- **NVML is broken on this box and `nvidia-smi` does not run.** The loaded kernel module is 580.126.09 while the userspace libraries are 580.173.02. CUDA is unaffected—`cuInit` returns 0, both devices enumerate and compute—but NVML-based utilisation and temperature logging is unavailable until the modules are reloaded, which needs root. Recorded because the run config is supposed to carry the driver version, and the number `nvidia-smi` would have reported cannot be obtained here.
- **Pipeline reproduced exactly from a clean tree.** `phase1_acquire.py` assembled **7,108,483** unique upper-triangle band entries with all three auxiliary md5s verified; `phase1_features.py` reproduced insulation $r = +0.9969$ ($n = 21{,}740$) and compartments $r = +0.9759$ ($n = 22{,}250$) against 4DN's own tracks, with 21,519/27,679 bins carrying a complete ϕ; `phase1_dataset.py` built **5,422 / 273 / 242** train/val/test windows with drop counts 1,054 (N fraction), 799 (structural coverage), 126 (buffer), 2 (short). Every one of these matches `docs/data_card.md` exactly, so the data layer is confirmed reproducible on a second machine rather than only on the machine that built it.
- `data/pilot_manifest.json` **now differs from the committed copy, and none of the differences are content.** The committed manifest was written on Windows. Rebuilding on Linux changed (i) path separators, `data\raw\` to `data/raw/`; (ii) the byte size of the two text files, because the Windows copies had CRLF line endings —`chr9.fa` shrank by exactly 2,306,580 bytes against 2,306,580 lines, and the GTF by exactly 169,673 bytes against 169,673 lines, one byte per line in each case; and (iii) the md5 of the two `.npz` files, whose byte sizes are nevertheless *identical*, because a zip central directory records the creating platform—`create_system` is 3 (Unix) here and 0 (Windows) there. The md5s of the three files downloaded from 4DN, which are the ones checked against the 4DN API, are unchanged. Band contents re-verified directly: 7,108,483 entries, 0 non-finite, min 4.25×10^{-6} / median 3.10×10^{-4} / max 0.942, 27,679 bins, 5,345 (19.31%) without a balancing weight—every figure matching `docs/data_card.md` §4. Whether to commit the Linux manifest is a judgement call left to the PI; a manifest whose md5s are platform-dependent cannot serve as a cross-platform integrity check for the derived files, only for the downloaded ones.
- `test_model.py`**: 16/16 passed** on this box, unchanged.
- **Triton kernel executed for the first time and validated: 34/34 checks passed** (`validate_kernel.py`, exit 0) across no-p, with-p, multi-chunk and ragged-final-chunk configurations, including the assertion that $\partial L/\partial p \neq 0$.
- **Three kernel defects fixed, all in the Triton lowering rather than the mathematics.** The hand-derived adjoint was correct as written. (i) `CHUNK` was a module-level global, which Triton refuses to read from a `@jit` function; it is now a `tl.constexpr` kernel parameter. (ii) The chunk walk used `range(NCHUNK-1,-1,-1)`; Triton lowers `range` to `scf.for`, which requires a non-negative step, so both reversed loops now count up and invert the index by hand. (iii) The inner loops used `tl.minimum(CHUNK, L-t0)` as a trip count; they now always run `CHUNK` times and predicate on $t < L$.
- **The backward is not bitwise reproducible.** `dBmat`, `dCmat` and `dp` are accumulated with `tl.atomic_add` across the d_{inner} programs, so their summation order varies between runs. Measured relative spread over five repeats at $B = 2$, $D = 512$, $L = 512$: 1.0×10^{-6},

9.6×10^{-7} and 8.9×10^{-7} respectively; `du`, `ddelta`, `dA` and `dDskip` were bitwise identical. This is fp32 rounding noise, not a defect, but it means seed-identical runs reproduce to fp32 tolerance and not to the bit. The paper must not claim bit-exact reproducibility.

- **Kernel checked beyond the supplied harness**, which runs at $D = 64$. Added checks at the real width $d_{\text{inner}} = 512$, at 64 chunks, and at $L = 1$ and $L = 65$; all agree with the reference. Width matters because the cross-channel atomics only contend at width.
- **Scan checkpoint memory measured**, since it bounds the Phase 3 batch size: 16.0 MiB per sample per scan at $d_{\text{inner}} = 512$, $d_{\text{state}} = 16$, $L = 32{,}768$, which is 0.50 GiB per sample across 16 layers and 2 directions, or 4.0 GiB at batch 8 before any other activation.
- **TeX Live installed user-local** (TinyTeX). The box had no LaTeX engine and no root, so `build_project_doc.py` could not run at all.

2026-08-10 (second entry) — Phase 3 training harness

`scripts/train.py` written. Results are recorded in a later entry; this one covers the harness and the platform findings only.

- **NCCL cannot be used on this box.** It calls `nvmlInit_v2()` during topology discovery, which fails for the same driver mismatch that breaks `nvidia-smi`. `init_process_group("nccl")` is lazy and succeeds anyway, so the failure surfaces at the first collective inside DDP's constructor. The script therefore probes NVML directly and falls back to **gloo**. Measured cost of the fallback: none detectable—two GPUs give 2.00× the single-GPU throughput (0.756 windows/s on one L40S, 1.51 windows/s on two).
- **PyTorch's CUDA caching allocator also calls NVML** when it reports memory pressure, so an out-of-memory condition surfaces as `INTERNAL ASSERT FAILED ... nvmlInit_v2_` rather than as `CUDA out of memory`. Anyone reading that traceback later should read it as OOM.
- **Kernel made 1.42× faster, and re-validated.** The scan's state vector is $d_{\text{state}} = 16$ elements wide, so Triton's default `num_warps=4` gave each program 128 lanes for 16 lanes of work. Setting `num_warps=1` took the real config from 7.55 to 5.30 s/step. Per-step losses were identical to the default across the benchmark, and `validate_kernel.py` was re-run afterwards: 34/34.
- **Gradient checkpointing implemented and then measured not to help.** The hypothesis was that the scan is latency-bound and a bigger batch bought with recompute would be nearly free. Batch scaling turned out to be linear—the GPU is already saturated at batch 2—so checkpointing costs its recompute undiluted: 1.32 s/window at batch 2 without it against 1.49 s/window at batch 8 with it. The code is retained behind `--grad-checkpoint`, off by default, as the lever to reach for if d_{model} or the window grows.
- **fp32 throughout, no autocast**, because the scan was validated in fp32 only. Training the experiment in bf16 would put it back on an unvalidated numeric path through a hand-written recurrence.
- **The baseline runs the custom Triton scan** even though it never supplies p, so the matched-compute claim against the structural arm is real rather than nominal.
- **N is excluded from masking and from the loss.** 12.00% of chr9 is N and a window may carry up to 10%. Including N would deflate bits/nucleotide by an amount that varies with each window's N content. The reported metric is therefore bits per *resolved* nucleotide, and its uniform-random floor is exactly 2.000 bits.
- **τ is measured two ways and they are not interchangeable.** `BiMambaLM.tau_stats()` derives τ from `dt_proj.bias` alone. That is a fair proxy at initialisation, where `dt_proj.weight` is small, but after training Δ is dominated by the input-dependent term $W_{dt}\delta'_t$, and a bias-only τ can be badly wrong. The script logs `tau_stats()` because the runbook asks for it,

and separately measures *empirical* τ from the Δ actually produced on validation batches. **The F4 gate must be read off the empirical numbers.**

- `num_workers` **defaults to 0.** `WindowDataset` owns an `np.random.default_rng` for reverse-complement augmentation; under multiple loader workers each worker copies it, so the augmentation stream would depend on how the sampler sharded and seeds would stop being reproducible.

2026-08-07

- **Phase 0 completed.** Eleven papers mapped. Two competitors found that were not on the original reading list: Evo2HiC, which is closer to the thesis than CHROME, and Lee's Evo 2 probe, which supports it.
- **Differentiator retired.** "Structure at training, sequence-only at inference" was dropped as a claim against Evo2HiC, whose DNA-only encoder has the same property. It still holds against CHROME.
- **Forward-citation sweep.** 318 papers checked; no work probes Akita or Orca representations on a non-folding task. The transfer gap stands.
- **Mechanism (a) selected**; (b) and (c) documented but not pursued.
- **Memory horizon measured.** $\tau_{\max} = 994$ tokens against a 5,000-token bin. Consequence: ϕ interpolation became mandatory, and a trained-τ measurement was added as a Phase 4 gate.
- **Pilot acquired and validated.** Insulation $r = +0.9969$ and compartments $r = +0.9759$ against independent 4DN tracks. A TAD and a ChIA-PET-corroborated loop confirmed visually.
- **Three assembly traps caught.** A 1,425 kb "loop" that was sparse-corner noise; an hg19 *ABL1* coordinate used to choose a GRCh38 region; and the standard GEO loop list for this dataset, which is hg19—detected because its largest chr9 anchor exceeds the length of GRCh38 chr9.
- **Dataset layer built.** Splits verified exact after a first version allowed held-out windows to straddle their region edge.
- **Model implemented; two bugs found by unit tests.** The reverse-complement sign flip was being applied to the encoded signal rather than to raw ϕ, where the encoder has already mixed all eight coordinates, making the operation meaningless. Separately, A was broadcast by trailing-axis alignment, which matches the state dimension against sequence length and only agrees when $L = d_{\text{inner}}$; the first test passed only because it used $L = 64$ with $d_{\text{inner}} = 64$.
- b_g **changed from** -8 **to** -4**.** The gradient through softplus is $\sigma(b_g)$, so -8 attenuated the gate's learning signal roughly 3000-fold—failure mode F2, self-inflicted by our own initialisation. Measured effect on the gate gradient: $2.06 \times 10^{-7} \to 7.69 \times 10^{-6}$.
- **Encoder bottleneck identified.** $\partial L/\partial s = W_{\Delta s}^{\top} \partial L/\partial \Delta$ with $W_{\Delta s}$ zero-initialised, so the encoder receives exactly zero gradient on step zero and roughly 10^{-9} afterwards against 10^{-4} for the matrix in front of it. Recommendation: remove the encoder and feed the eight standardised features directly at $d_s = 8$, costing $+1.700\%$ parameters and eliminating F8.
- **Triton kernel written** for the permeability term, with a GPU validation harness. Unverified until executed.

9 Open items

- **Decide whether bit-exact reproducibility is required.** If it is, the atomics in the backward must be replaced with a deterministic reduction, at a performance cost. If it

is not—the reasonable reading, given that the Phase 5 gate is across-seed variance rather than a single number—say so explicitly, because the spread is measured and small.

- **Restore** `nvidia-smi` by reloading the NVIDIA kernel modules, or accept that no driver-level utilisation figure can be logged alongside the runs.
- **Confirm the encoder removal** ($d_s = 8$, no encoder), which changes parameter figures recorded in three documents.
- **Read CHROME line by line.** The summary here came from automated extraction of the PMC full text.
- **Obtain Orca's Results section**, paywalled at *Nature Genetics* with no PubMed Central deposit. The claim that its representations were never evaluated on a non-folding task rests on the abstract, all ten extended-data captions and the section headings, not the body.
- **Re-check Evo2HiC** for a revision adding variant evaluation before Phase 4 commits GPU time; that would close the one contribution nothing currently contests.
- **Re-check Lee's preprint** for peer-review status before it carries weight in a methods section.

10 What happens next

Phase 3 trains the sequence-only baseline, logging every hyperparameter, seed and metric before any structural model exists, so no flattering comparison can be selected after the fact. That run must also report the trained memory horizon, which decides whether a model of this size can represent TAD-scale dependencies at all. If it cannot, the mechanism is re-scoped to sub-TAD structure or the model is enlarged, and that decision is taken before Phase 4 rather than after.

Every measured number in this document traces to a named script: `phase1_acquire.py`, `phase1_features.py`, `phase1_validate_visual.py`, `phase1_dataset.py`, `param_accounting.py`, `f4_memory_horizon.py`, `test_model.py`. Literature claims are sourced in `docs/related_work.md`; the mechanism is specified in `docs/architecture_spec.md`; data provenance is in `docs/data_card.md`.